

Helix

Docs for bleeding edge master can be found at <https://docs.helix-editor.com/master>.

See the [usage](#) section for a quick overview of the editor, [keymap](#) section for all available keybindings and the [configuration](#) section for defining custom keybindings, setting themes, etc. For everything else (e.g., how to install supported language servers), see the [Helix Wiki](#).

Refer the [FAQ](#) for common questions.

Installing Helix

The typical way to install Helix is via your operating system's package manager.

Note that:

- To get the latest nightly version of Helix, you need to [build from source](#).
- To take full advantage of Helix, install the language servers for your preferred programming languages. See the [wiki](#) for instructions.

Pre-built binaries

Download pre-built binaries from the [GitHub Releases](#) page. The tarball contents include an `hx` binary and a `runtime` directory. To set up Helix:

1. Add the `hx` binary to your system's `$PATH` to allow it to be used from the command line.
2. Copy the `runtime` directory to a location that `hx` searches for runtime files. A typical location on Linux/macOS is `~/.config/helix/runtime`.

To see the runtime directories that `hx` searches, run `hx --health`. If necessary, you can override the default runtime location by setting the `HELIX_RUNTIME` environment variable.

Package managers

- Linux
 - Ubuntu/Debian
 - Ubuntu (PPA)
 - Fedora/RHEL
 - Arch Linux extra
 - NixOS
 - Flatpak
 - Snap
 - AppImage
- macOS
 - Homebrew Core
 - MacPorts
- Windows
 - Winget
 - Scoop
 - Chocolatey
 - MSYS2

Packaging status	
Alpine Linux 3.16	22.03
Alpine Linux 3.17	22.08.1
Alpine Linux 3.18	23.03
Alpine Linux 3.19	23.10
Alpine Linux 3.20	24.03
Alpine Linux 3.21	24.07
Alpine Linux 3.22	25.01.1
Alpine Linux 3.23	25.07.1
Alpine Linux Edge	25.07.1
ALT Linux p10	24.07
ALT Linux p11	25.07.1
ALT Sisyphus	25.07.1
AOSC	25.07.1
Arch Linux	25.07.1
AUR	25.07.r532.g...
Artix	25.07.1
Baulk	25.07.1
Chimera Linux	25.07.1
Chocolatey	25.7.1
Chromebrew	22.12
Debian 13	25.01.1
Debian 14	25.07.1+2026...
Debian Unstable	25.07.1+2026...
Devuan Unstable	25.07.1+2026...
EPEL 9	25.07.1
EPEL 10	25.07.1
Fedora 38	24.03
Fedora 39	24.07
Fedora 40	25.01.1
Fedora 41	25.07.1
Fedora 42	25.07.1
Fedora 43	25.07.1
Fedora 44	25.07.1
Fedora Rawhide	25.07.1
FreeBSD Ports	25.07.1
Gentoo	25.07.1
GNU Guix	25.07.1
Homebrew	25.07.1
Kali Linux Rolling	25.07.1+2026...
KISS Community - community	25.07.1
LiGurOS stable	25.07.1
LiGurOS develop	25.07.1
MacPorts	25.07.1
Mageia 9	23.05
Mageia cauldron	24.03
Manjaro Stable	25.07.1
Manjaro Testing	25.07.1

Manjaro Unstable	25.07.1
MSYS2 clang64	25.07.1
MSYS2 clangarm64	25.07.1
MSYS2 mingw	25.07.1
MSYS2 ucrt64	25.07.1
nixpkgs stable 23.11	23.10
nixpkgs stable 24.05	24.03
nixpkgs stable 24.11	25.01.1
nixpkgs stable 25.05	25.07
nixpkgs stable 25.11	25.07.1
nixpkgs unstable	25.07.1
OpenBSD Ports	25.07.1
OpenIndiana packages	25.07.1
openSUSE Leap 15.5	23.03
openSUSE Leap 15.6	23.03
openSUSE Tumbleweed	25.07.1
pacstall	25.07.1
Parabola	25.07.1
Parrot	25.01.1
Pisi Linux	25.07.1
pkgsrc-2025Q4	23.10
pkgsrc-2026Q1	25.07.1
pkgsrc current	25.07.1
PureOS landing	25.01.1
Raspbian Stable	25.01.1
Raspbian Testing	25.07.1+2026...
Scoop	25.07.1
SlackBuilds	25.07.1
Solus	25.07.1
T2 SDE	25.07.1
Termux	25.07.1
Ubuntu 26.04	25.07.1+2025...
Ubuntu 26.10	25.07.1+2025...
Void Linux x86_64	25.07.1
Wikidata	25.07.1

Linux

The following third party repositories are available:

Ubuntu/Debian

Install the Debian package from the release page.

If you are running a system older than Ubuntu 22.04, Mint 21, or Debian 12, you can build the `.deb` file locally from source.

Ubuntu (PPA)

Add the `PPA` for Helix:

```
sudo add-apt-repository ppa:maveonair/helix-editor
sudo apt update
sudo apt install helix
```

Fedora/RHEL

```
sudo dnf install helix
```

Arch Linux extra

Releases are available in the `extra` repository:

```
sudo pacman -S helix
```

💡 When installed from the `extra` repository, run Helix with `helix` instead of `hx`.

For example:

```
helix --health
```

to check health

Additionally, a `helix-git` package is available in the AUR, which builds the master branch.

NixOS

Helix is available in `nixpkgs` through the `helix` attribute, the unstable channel usually carries the latest release.

Helix is also available as a `flake` in the project root. Use `nix develop` to spin up a reproducible development shell. Outputs are cached for

each push to master using `Cachix`. The flake is configured to automatically make use of this cache assuming the user accepts the new settings on first use.

If you are using a version of Nix without flakes enabled, install `Cachix CLI` and use `cachix use helix` to configure Nix to use cached outputs when possible.

Flatpak

Helix is available on Flathub:

```
flatpak install flathub com.helix_editor.Helix
flatpak run com.helix_editor.Helix
```

Snap

Helix is available on Snapcraft and can be installed with:

```
snap install --classic helix
```

This will install Helix as both `/snap/bin/helix` and `/snap/bin/hx`, so make sure `/snap/bin` is in your `PATH`.

AppImage

Install Helix using the Linux AppImage format. Download the official Helix AppImage from the [latest releases page](#).

```
chmod +x helix-*.AppImage # change permission for executable mode
./helix-*.AppImage # run helix
```

You can optionally add the `.desktop` file. Helix must be installed in `PATH` with the name `hx`. For example:

```
mkdir -p "$HOME/.local/bin"
mv helix-*.AppImage "$HOME/.local/bin/hx"
```

and make sure `~/.local/bin` is in your `PATH`.

macOS

Homebrew Core

```
brew install helix
```

MacPorts

```
sudo port install helix
```

Windows

Install on Windows using Winget, Scoop, Chocolatey or MSYS2.

Winget

Windows Package Manager winget command-line tool is by default available on Windows 11 and modern versions of Windows 10 as a part of the App Installer. You can get App Installer from the Microsoft Store. If it's already installed, make sure it is updated with the latest version.

```
winget install Helix.Helix
```

Scoop

```
scoop install helix
```

Chocolatey

```
choco install helix
```


MSYS2

For 64-bit Windows 8.1 or above:

```
pacman -S mingw-w64-ucrt-x86_64-helix
```

Building from source

- Configuring Helix's runtime files
 - Linux and macOS
 - Windows
 - Multiple runtime directories
 - Note to packagers
- Validating the installation
- Configure the desktop shortcut
- Building the Debian package

Requirements:

Clone the Helix GitHub repository into a directory of your choice. The examples in this documentation assume installation into either `~/src/` on Linux and macOS, or `%userprofile%\src\` on Windows.

- The Rust toolchain
- The Git version control system
- A C++14 compatible compiler to build the tree-sitter grammars, for example GCC or Clang

If you are using the `musl-libc` standard library instead of `glibc` the following environment variable must be set during the build to ensure tree-sitter grammars can be loaded correctly:

```
RUSTFLAGS="-C target-feature=--crt-static"
```

1. Clone the repository:

```
git clone https://github.com/helix-editor/helix
cd helix
```

2. Compile from source:

```
# Reproducible
cargo install --path helix-term --locked
```

```
# Optimized
cargo install \
  --profile opt \
  --config 'build.rustflags="-C target-cpu=native"' \
  --path helix-term \
  --locked
```

Either command will create the `hx` executable and construct the tree-sitter grammars in the local `runtime` folder.

💡 If you do not want to fetch or build grammars, set an environment variable `HELIX_DISABLE_AUTO_GRAMMAR_BUILD`

💡 Tree-sitter grammars can be fetched and compiled if not pre-packaged. Fetch grammars with `hx --grammar fetch` and compile them with `hx --grammar build`. This will install them in the `runtime` directory within the user's helix config directory (more details below).

Configuring Helix's runtime files

Linux and macOS

The `runtime` directory is one below the Helix source, so either export a `HELIX_RUNTIME` environment variable to point to that directory and add it to your `~/.bashrc` or equivalent:

```
export HELIX_RUNTIME=~/.src/helix/runtime
```

Or, create a symbolic link:

```
ln -Ts $PWD/runtime ~/.config/helix/runtime
```

Windows

Either set the `HELIX_RUNTIME` environment variable to point to the runtime files using the Windows setting (search for `Edit environment`

variables for your account) or use the `setx` command in Cmd:

```
setx HELIX_RUNTIME "%userprofile%\src\helix\runtime"
```

💡 `%userprofile%` resolves to your user directory like `C:\Users\Your-Name\` for example.

Or, create a symlink in `%appdata%\helix\` that links to the source code directory:

Method	Command
PowerShell	<pre>New-Item -ItemType Junction -Target "runtime" -Path "\$Env:AppData\helix\runtime"</pre>
Cmd	<pre>cd %appdata%\helix mklink /D runtime "%userprofile%\src\helix\runtime"</pre>

💡 On Windows, creating a symbolic link may require running PowerShell or Cmd as an administrator.

Multiple runtime directories

When Helix finds multiple runtime directories it will search through them for files in the following order:

1. `runtime/` sibling directory to `$CARGO_MANIFEST_DIR` directory (this is intended for developing and testing helix only).
2. `runtime/` subdirectory of OS-dependent helix user config directory.
3. `$HELIX_RUNTIME`
4. Distribution-specific fallback directory (set at compile time—not run time—with the `HELIX_DEFAULT_RUNTIME` environment variable)
5. `runtime/` subdirectory of path to Helix executable.

This order also sets the priority for selecting which file will be used if multiple runtime directories have files with the same name.

Note to packagers

If you are making a package of Helix for end users, to provide a good

out of the box experience, you should set the `HELIX_DEFAULT_RUNTIME` environment variable at build time (before invoking `cargo build`) to a directory which will store the final runtime files after installation. For example, say you want to package the runtime into `/usr/lib/helix/runtime`. The rough steps a build script could follow are:

1. `export HELIX_DEFAULT_RUNTIME=/usr/lib/helix/runtime`
2. `cargo build --profile opt --locked`
3. `cp -r runtime $BUILD_DIR/usr/lib/helix/`
4. `cp target/opt/hx $BUILD_DIR/usr/bin/hx`

This way the resulting `hx` binary will always look for its runtime directory in `/usr/lib/helix/runtime` if the user has no custom runtime in `~/.config/helix` or `HELIX_RUNTIME`.

Validating the installation

To make sure everything is set up as expected you should run the Helix health check:

```
hx --health
```

For more information on the health check results refer to [Health check](#).

Configure the desktop shortcut

If your desktop environment supports the `XDG desktop menu` you can configure Helix to show up in the application menu by copying the provided `.desktop` and icon files to their correct folders:

```
cp contrib/Helix.desktop ~/.local/share/applications
cp contrib/helix.png ~/.icons # or ~/.local/share/icons
```

It is recommended to convert the links in the `.desktop` file to absolute paths to avoid potential problems:

```
sed -i -e "s|Exec=hx %F|Exec=$(readlink -f ~/.cargo/bin/hx) %F|g" \
-e "s|Icon=helix|Icon=$(readlink -f ~/.icons/helix.png)|g" ~/.local/share/
applications/Helix.desktop
```

To use another terminal than the system default, you can modify the `.desktop` file. For example, to use `kitty`:

```
sed -i "s|Exec=hx %F|Exec=kitty hx %F|g" ~/.local/share/applications/  
Helix.desktop  
sed -i "s|Terminal=true|Terminal=false|g" ~/.local/share/applications/  
Helix.desktop
```

Building the Debian package

If the `.deb` file provided on the release page uses a `libc` version higher than that used by your Debian, Ubuntu, or Mint system, you can build the package from source to match your system's dependencies.

Install `cargo-deb`, the tool used for building the `.deb` file:

```
cargo install cargo-deb
```

After cloning and entering the Helix repository as previously described, use the following command to build the release binary and package it into a `.deb` file in a single step.

```
cargo deb -- --locked
```

💡 This locks you into the `--release` profile. But you can also build helix in any way you like. As long as you leave a `target/release/hx` file, it will get packaged with `cargo deb --no-build`

💡 Don't worry about the following:

```
warning: Failed to find dependency specification
```

Cargo deb just reports which packaged files it didn't derive dependencies for. But so far the dependency deriving seems very good, even if some of the grammar files are skipped.

You can find the resulted `.deb` in `target/debian/`. It should contain everything it needs, including the

- completions for bash, fish, zsh
- .desktop file
- icon (though desktop environments might use their own since the name of the package is correctly `helix`)
- launcher to the binary with the runtime

Using Helix

For a full interactive introduction to Helix, refer to the `tutor` which can be accessed via the command `hx --tutor` or `:tutor`.

💡 Currently, not all functionality is fully documented, please refer to the `key mappings` list.

Modes

Helix is a modal editor, meaning it has different modes for different tasks. The main modes are:

- **Normal mode**: For navigation and editing commands. This is the default mode.
- **Insert mode**: For typing text directly into the document. Access by typing `i` in normal mode.
- **Select/extend mode**: For making selections and performing operations on them. Access by typing `v` in normal mode.

Buffers

Buffers are in-memory representations of files. You can have multiple buffers open at once. Use `pickers` or commands like `:buffer-next` and `:buffer-previous` to open buffers or switch between them.

Selection-first editing

Inspired by `Kakoune`, Helix follows the `selection → action` model. This means that whatever you are going to act on (a word, a paragraph, a line, etc.) is selected first and the action itself (delete, change, yank, etc.) comes second. A cursor is simply a single width selection.

Multiple selections

Also inspired by Kakoune, multiple selections are a core mode of interaction in Helix. For example, the standard way of replacing multiple instances of a word is to first select all instances (so there is one selection per instance) and then use the change action (`c`) to edit them all at the same time.

Motions

Motions are commands that move the cursor or modify selections. They're used for navigation and text manipulation. Examples include `w` to move to the next word, or `f` to find a character. See the [Movement](#) section of the keymap for more motions.

Registers

- User-defined registers
- Default registers
- Special registers

In Helix, registers are storage locations for text and other data, such as the result of a search. Registers can be used to cut, copy, and paste text, similar to the clipboard in other text editors. Usage is similar to Vim, with `"` being used to select a register.

User-defined registers

Helix allows you to create your own named registers for storing text, for example:

- `"ay` - Yank the current selection to register `a`.
- `"op` - Paste the text in register `o` after the selection.

If a register is selected before invoking a change or delete command, the selection will be stored in the register and the action will be carried out:

- `"hc` - Store the selection in register `h` and then change it (delete and enter insert mode).
- `"md` - Store the selection in register `m` and delete it.

Default registers

Commands that use registers, like yank (`y`), use a default register if none is specified. These registers are used as defaults:

Register character	Contains
<code>/</code>	Last search
<code>:</code>	Last executed command
<code>"</code>	Last yanked text
<code>@</code>	Last recorded macro

Special registers

Some registers have special behavior when read from and written to.

Register character	When read	When written
<code>_</code>	No values are returned	All values are discarded
<code>#</code>	Selection indices (first selection is <code>1</code> , second is <code>2</code> , etc.)	This register is not writable
<code>.</code>	Contents of the current selections	This register is not writable
<code>%</code>	Name of the current file	This register is not writable
<code>+</code>	Reads from the system clipboard	Joins and yanks to the system clipboard
<code>*</code>	Reads from the primary clipboard	Joins and yanks to the primary clipboard

When yanking multiple selections to the clipboard registers, the selections are joined with newlines. Pasting from these registers will paste multiple selections if the clipboard was last yanked to by the Helix session. Otherwise the clipboard contents are pasted as one selection.

Surround

Helix includes built-in functionality similar to `vim-surround`. The keymappings have been inspired from `vim-sandwich`:

```
1 print("some (inside) text")
2 print("other (inside) long text")
3
```

Key Sequence	Action
<code>ms<char></code> (after selecting text)	Add surround characters to selection
<code>mr<char_to_replace><new_char></code>	Replace the closest surround characters
<code>md<char_to_delete></code>	Delete the closest surround characters

You can use counts to act on outer pairs.

Surround can also act on multiple selections. For example, to change every occurrence of `(use)` to `[use]`:

1. `%` to select the whole file
2. `s` to split the selections on a search term
3. Input `use` and hit Enter
4. `mr([` to replace the parentheses with square brackets

Multiple characters are currently not supported, but planned for future release.

Selecting and manipulating text with textobjects

In Helix, textobjects are a way to select, manipulate and operate on a piece of text in a structured way. They allow you to refer to blocks of text based on their structure or purpose, such as a word, sentence, paragraph, or even a function or block of code.

```
2 (some text) with words {and} surrounds
3
```

```
6
5 class Class:
4     def __init__(self):
3         self.x = 12
2
1     def do_something(self):
7         print("Hello world")
1         print(self.x)
~
```

- `ma` - Select around the object (`va` in Vim, `<alt-a>` in Kakoune)
- `mi` - Select inside the object (`vi` in Vim, `<alt-i>` in Kakoune)

Key after <code>mi</code> or <code>ma</code>	Textobject selected
<code>w</code>	Word
<code>W</code>	WORD
<code>p</code>	Paragraph
<code>(</code> , <code>[</code> , <code>'</code> , etc.	Specified surround pairs
<code>m</code>	The closest surround pair
<code>f</code>	Function
<code>t</code>	Type (or Class)

Key after <code>mi</code> or <code>ma</code>	Textobject selected
<code>a</code>	Argument/parameter
<code>c</code>	Comment
<code>T</code>	Test
<code>g</code>	Change

💡 `f`, `t`, etc. need a tree-sitter grammar active for the current document and a special tree-sitter query file to work properly. Only some grammars currently have the query file implemented. Contributions are welcome!

Navigating using tree-sitter textobjects

Navigating between functions, classes, parameters, and other elements is possible using tree-sitter and textobject queries. For example to move to the next function use `]f`, to move to previous type use `[t`, and so on.

```
2 class Class:
1     def __init__(self):
2         self.x = 10
3
4     def method(self):
5         print(self.x)
6         self.x += 1
7
8 if __name__ == "__main__":
9     pass
```

For the full reference see the [unpaired](#) section of the [key bind documentation](#).

💡 This feature relies on tree-sitter textobjects and requires the corresponding query file to work properly.

Moving the selection with syntax-aware motions

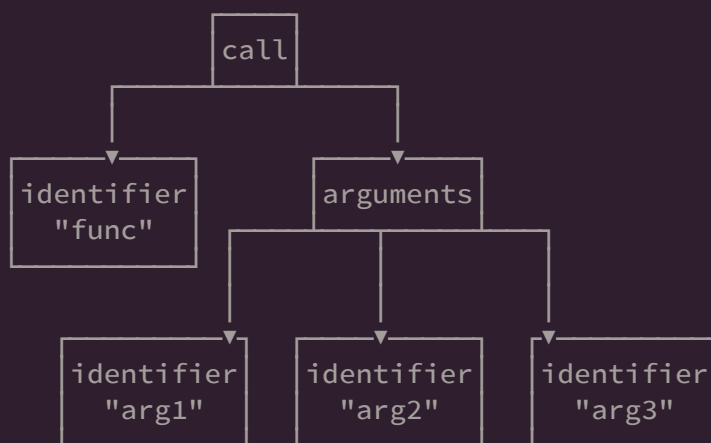
Alt-p, **Alt-o**, **Alt-i**, and **Alt-n** (or **Alt** and arrow keys) allow you to move the selection according to its location in the syntax tree. For example, many languages have the following syntax for function calls:

```
func(arg1, arg2, arg3);
```

A function call might be parsed by tree-sitter into a tree like the following.

```
(call
  function: (identifier) ; func
  arguments:
    (arguments      ; (arg1, arg2, arg3)
      (identifier)  ; arg1
      (identifier)  ; arg2
      (identifier)) ; arg3)
```

Use **:tree-sitter-subtree** to view the syntax tree of the primary selection. In a more intuitive tree format:



If you have a selection that wraps **arg1** (see the tree above), and you use **Alt-n**, it will select the next sibling in the syntax tree: **arg2**.


```
// before  
func([arg1], arg2, arg3)  
// after  
func(arg1, [arg2], arg3);
```

Similarly, **Alt-o** will expand the selection to the parent node, in this case, the arguments node.

```
func([arg1, arg2, arg3]);
```

There is also some nuanced behavior that prevents you from getting stuck on a node with no sibling. When using **Alt-p** with a selection on **arg1**, the previous child node will be selected. In the event that **arg1** does not have a previous sibling, the selection will move up the syntax tree and select the previous element. As a result, using **Alt-p** with a selection on **arg1** will move the selection to the "func" **identifier**.

Using pickers

Helix has a variety of pickers, which are interactive windows used to select various kinds of items. These include a file picker, global search picker, and more. Most pickers are accessed via keybindings in `space mode`. Pickers have their own `keymap` for navigation.

Filtering Picker Results

Most pickers perform fuzzy matching using `fzf syntax`. Two exceptions are the global search picker, which uses `regex`, and the workspace symbol picker, which passes search terms to the language server. Note that OR operations (`|`) are not currently supported.

If a picker shows multiple columns, you may apply the filter to a specific column by prefixing the column name with `%`. Column names can be shortened to any prefix, so `%p`, `%pa` or `%pat` all mean the same as `%path`. For example, a query of `helix %p .toml !lang` in the global search picker searches for the term "helix" within files with paths ending in ".toml" but not including "lang".

You can insert the contents of a `register` using `Ctrl-r` followed by a register name. For example, one could insert the currently selected text using `Ctrl-r-.`, or the directory of the current file using `Ctrl-r-%` followed by `Ctrl-w` to remove the last path section. The global search picker will use the contents of the `search register` if you press `Enter` without typing a filter. For example, pressing `*-Space-/-Enter` will start a global search for the currently selected text.

Keymap

- Normal mode
 - Movement
 - Changes
 - Shell
 - Selection manipulation
 - Search
 - Minor modes
 - View mode
 - Goto mode
 - Match mode
 - Window mode
 - Space mode
 - Popup
 - Completion Menu
 - Signature-help Popup
 - Unimpaired
- Insert mode
- Select / extend mode
- Picker
- Prompt

💡 Mappings marked (LSP) require an active language server for the file.

💡 Mappings marked (TS) require a tree-sitter grammar for the file type.

⚠ Some terminals' default key mappings conflict with Helix's. If any of the mappings described on this page do not work as expected, check your terminal's mappings to ensure they do not conflict. See the [wiki](#) for known conflicts.

Normal mode

Normal mode is the default mode when you launch helix. You can return to it from other modes by pressing the `Escape` key.

Movement

NOTE: Unlike Vim, `f`, `F`, `t` and `T` are not confined to the current line.

Key	Description	Command
<code>h</code> , <code>Left</code>	Move left	<code>move_char_left</code>
<code>j</code> , <code>Down</code>	Move down	<code>move_visual_line_down</code>
<code>k</code> , <code>Up</code>	Move up	<code>move_visual_line_up</code>
<code>l</code> , <code>Right</code>	Move right	<code>move_char_right</code>
<code>w</code>	Move next word start	<code>move_next_word_start</code>
<code>b</code>	Move previous word start	<code>move_prev_word_start</code>
<code>e</code>	Move next word end	<code>move_next_word_end</code>
<code>W</code>	Move next WORD start	<code>move_next_long_word_start</code>
<code>B</code>	Move previous WORD start	<code>move_prev_long_word_start</code>
<code>E</code>	Move next WORD end	<code>move_next_long_word_end</code>
<code>t</code>	Find till next char	<code>find_till_char</code>
<code>f</code>	Find next char	<code>find_next_char</code>
<code>T</code>	Find till previous char	<code>till_prev_char</code>
<code>F</code>	Find previous char	<code>find_prev_char</code>
<code>G</code>	Go to line number <code><n></code>	<code>goto_line</code>
<code>Alt-.</code>	Repeat last motion (<code>f</code> , <code>t</code> , <code>m</code> , <code>[</code> or <code>]</code>)	<code>repeat_last_motion</code>
<code>Home</code>	Move to the start of the line	<code>goto_line_start</code>
<code>End</code>	Move to the end of the line	<code>goto_line_end</code>

Key	Description	Command
<code>Ctrl-b</code> , <code>PageUp</code>	Move page up	<code>page_up</code>
<code>Ctrl-f</code> , <code>PageDown</code>	Move page down	<code>page_down</code>
<code>Ctrl-u</code>	Move cursor and page half page up	<code>page_cursor_half_up</code>
<code>Ctrl-d</code>	Move cursor and page half page down	<code>page_cursor_half_down</code>
<code>Ctrl-i</code>	Jump forward on the jumplist	<code>jump_forward</code>
<code>Ctrl-o</code>	Jump backward on the jumplist	<code>jump_backward</code>
<code>Ctrl-s</code>	Save the current selection to the jumplist	<code>save_selection</code>

Changes

Key	Description	Command
<code>r</code>	Replace with a character	<code>replace</code>
<code>R</code>	Replace with yanked text	<code>replace_with_yanked</code>
<code>~</code>	Switch case of the selected text	<code>switch_case</code>
<code>`</code>	Set the selected text to lower case	<code>switch_to_lowercase</code>
<code>Alt-`</code>	Set the selected text to upper case	<code>switch_to_uppercase</code>
<code>i</code>	Insert before selection	<code>insert_mode</code>
<code>a</code>	Insert after selection (append)	<code>append_mode</code>
<code>I</code>	Insert at the start of the line	<code>insert_at_line_start</code>
<code>A</code>	Insert at the end of the line	<code>insert_at_line_end</code>
<code>o</code>	Open new line below selection	<code>open_below</code>
<code>O</code>	Open new line above selection	<code>open_above</code>
<code>.</code>	Repeat last insert	N/A

Key	Description	Command
u	Undo change	<code>undo</code>
U	Redo change	<code>redo</code>
Alt-u	Move backward in history	<code>earlier</code>
Alt-U	Move forward in history	<code>later</code>
y	Yank selection	<code>yank</code>
p	Paste after selection	<code>paste_after</code>
P	Paste before selection	<code>paste_before</code>
" <reg>	Select a register to yank to or paste from	<code>select_register</code>
>	Indent selection	<code>indent</code>
<	Unindent selection	<code>unindent</code>
=	Format selection (LSP)	<code>format_selections</code>
d	Delete selection	<code>delete_selection</code>
Alt-d	Delete selection, without yanking	<code>delete_selection_noyank</code>
c	Change selection (delete and enter insert mode)	<code>change_selection</code>
Alt-c	Change selection (delete and enter insert mode, without yanking)	<code>change_selection_noyank</code>
Ctrl-a	Increment object (number) under cursor	<code>increment</code>
Ctrl-x	Decrement object (number) under cursor	<code>decrement</code>
Q	Start/stop macro recording to the selected register (experimental)	<code>record_macro</code>
q	Play back a recorded macro from the selected register (experimental)	<code>replay_macro</code>

Shell

Key	Description	Command
-----	-------------	---------

Key	Description	Command
	Pipe each selection through shell command, replacing with output	shell_pipe
Alt-	Pipe each selection into shell command, ignoring output	shell_pipe_to
!	Run shell command, inserting output before each selection	shell_insert_output
Alt-!	Run shell command, appending output after each selection	shell_append_output
\$	Pipe each selection into shell command, keep selections where command returned 0	shell_keep_pipe

Selection manipulation

Key	Description	Command
s	Select all regex matches inside selections	select_regex
S	Split selection into sub selections on regex matches	split_selection
Alt-s	Split selection on newlines	split_selection_on_newline
Alt-minus	Merge selections	merge_selections
Alt-_	Merge consecutive selections	merge_consecutive_selections
&	Align selection in columns	align_selections
-	Trim whitespace from the selection	trim_selections
;	Collapse selection onto a single cursor	collapse_selection
Alt-;	Flip selection cursor and anchor	flip_selections

Key	Description	Command
<code>Alt-:</code>	Ensures the selection is in forward direction	<code>ensure_selections_forward</code>
<code>,</code>	Keep only the primary selection	<code>keep_primary_selection</code>
<code>Alt-,</code>	Remove the primary selection	<code>remove_primary_selection</code>
<code>C</code>	Copy selection onto the next line (Add cursor below)	<code>copy_selection_on_next_line</code>
<code>Alt-C</code>	Copy selection onto the previous line (Add cursor above)	<code>copy_selection_on_prev_line</code>
<code>(</code>	Rotate main selection backward	<code>rotate_selections_backward</code>
<code>)</code>	Rotate main selection forward	<code>rotate_selections_forward</code>
<code>Alt-(</code>	Rotate selection contents backward	<code>rotate_selection_contents_backward</code>
<code>Alt-)</code>	Rotate selection contents forward	<code>rotate_selection_contents_forward</code>
<code>%</code>	Select entire file	<code>select_all</code>
<code>x</code>	Select current line, if already selected, extend to next line	<code>extend_line_below</code>
<code>X</code>	Extend selection to line bounds (line-wise selection)	<code>extend_to_line_bounds</code>
<code>Alt-x</code>	Shrink selection to line bounds (line-wise selection)	<code>shrink_to_line_bounds</code>
<code>J</code>	Join lines inside selection	<code>join_selections</code>
<code>Alt-J</code>	Join lines inside selection and select the	<code>join_selections_space</code>

Key	Description	Command
	inserted space	
<code>K</code>	Keep selections matching the regex	<code>keep_selections</code>
<code>Alt-K</code>	Remove selections matching the regex	<code>remove_selections</code>
<code>Ctrl-c</code>	Comment/uncomment the selections	<code>toggle_comments</code>
<code>Alt-o</code> , <code>Alt-up</code>	Expand selection to parent syntax node (TS)	<code>expand_selection</code>
<code>Alt-i</code> , <code>Alt-down</code>	Shrink syntax tree object selection (TS)	<code>shrink_selection</code>
<code>Alt-p</code> , <code>Alt-left</code>	Select previous sibling node in syntax tree (TS)	<code>select_prev_sibling</code>
<code>Alt-n</code> , <code>Alt-right</code>	Select next sibling node in syntax tree (TS)	<code>select_next_sibling</code>
<code>Alt-a</code>	Select all sibling nodes in syntax tree (TS)	<code>select_all_siblings</code>
<code>Alt-I</code> , <code>Alt-Shift-down</code>	Select all children nodes in syntax tree (TS)	<code>select_all_children</code>
<code>Alt-e</code>	Move to end of parent node in syntax tree (TS)	<code>move_parent_node_end</code>
<code>Alt-b</code>	Move to start of parent node in syntax tree (TS)	<code>move_parent_node_start</code>

Search

Search commands all operate on the `/` register by default. To use a different register, use `"<char>`.

Key	Description	Command
<code>/</code>	Search for regex pattern	<code>search</code>

Key	Description	Command
<code>?</code>	Search for previous pattern	<code>rsearch</code>
<code>n</code>	Select next search match	<code>search_next</code>
<code>N</code>	Select previous search match	<code>search_prev</code>
<code>*</code>	Use current selection as the search pattern, automatically wrapping with <code>\b</code> on word boundaries	<code>search_selection_detect_word_boundaries</code>
<code>Alt</code> <code>-*</code>	Use current selection as the search pattern	<code>search_selection</code>

Minor modes

These sub-modes are accessible from normal mode and typically switch back to normal mode after a command.

Key	Description	Command
<code>v</code>	Enter <code>select</code> (extend) mode	<code>select_mode</code>
<code>g</code>	Enter <code>goto</code> mode	N/A
<code>m</code>	Enter <code>match</code> mode	N/A
<code>:</code>	Enter <code>command</code> mode	<code>command_mode</code>
<code>z</code>	Enter <code>view</code> mode	N/A
<code>Z</code>	Enter <code>sticky view</code> mode	N/A
<code>Ctrl-w</code>	Enter <code>window</code> mode	N/A
<code>Space</code>	Enter <code>space</code> mode	N/A

These modes (except command mode) can be configured by remapping keys.

View mode

Accessed by typing `z` in normal mode.

View mode is intended for scrolling and manipulating the view without changing the selection. The "sticky" variant of this mode (accessed by typing `z` in normal mode) is persistent and can be exited using the escape key. This is useful when you're simply looking over text and not actively editing it.

Key	Description	Command
<code>z</code> , <code>c</code>	Vertically center the line	<code>align_view_center</code>
<code>t</code>	Align the line to the top of the screen	<code>align_view_top</code>
<code>b</code>	Align the line to the bottom of the screen	<code>align_view_bottom</code>
<code>m</code>	Align the line to the middle of the screen (horizontally)	<code>align_view_middle</code>
<code>j</code> , <code>down</code>	Scroll the view downwards	<code>scroll_down</code>
<code>k</code> , <code>up</code>	Scroll the view upwards	<code>scroll_up</code>
<code>Ctrl-f</code> , <code>PageDown</code>	Move page down	<code>page_down</code>
<code>Ctrl-b</code> , <code>PageUp</code>	Move page up	<code>page_up</code>
<code>Ctrl-u</code>	Move cursor and page half page up	<code>page_cursor_half_up</code>
<code>Ctrl-d</code>	Move cursor and page half page down	<code>page_cursor_half_down</code>

Goto mode

Accessed by typing `g` in normal mode.

Jumps to various locations.

Key	Description	Command
<code>g</code>	Go to line number <code><n></code> else start of file	<code>goto_file_start</code>
<code>l</code>	Go to column number <code><n></code> else start of line	<code>goto_column</code>
<code>e</code>	Go to the end of the file	<code>goto_last_line</code>

Key	Description	Command
f	Go to files in the selections	<code>goto_file</code>
h	Go to the start of the line	<code>goto_line_start</code>
l	Go to the end of the line	<code>goto_line_end</code>
s	Go to first non-whitespace character of the line	<code>goto_first_nonwhitespace</code>
t	Go to the top of the screen	<code>goto_window_top</code>
c	Go to the middle of the screen	<code>goto_window_center</code>
b	Go to the bottom of the screen	<code>goto_window_bottom</code>
d	Go to definition (LSP)	<code>goto_definition</code>
y	Go to type definition (LSP)	<code>goto_type_definition</code>
r	Go to references (LSP)	<code>goto_reference</code>
i	Go to implementation (LSP)	<code>goto_implementation</code>
a	Go to the last accessed/alternate file	<code>goto_last_accessed_file</code>
m	Go to the last modified/alternate file	<code>goto_last_modified_file</code>
n	Go to next buffer	<code>goto_next_buffer</code>
p	Go to previous buffer	<code>goto_previous_buffer</code>
.	Go to last modification in current file	<code>goto_last_modification</code>
j	Move down textual (instead of visual) line	<code>move_line_down</code>
k	Move up textual (instead of visual) line	<code>move_line_up</code>
w	Show labels at each word and select the word that belongs to the entered labels	<code>goto_word</code>

Match mode

Accessed by typing **m** in normal mode.

Please refer to the relevant sections for detailed explanations about `surround` and `textobjects`.

Key	Description	Command
<code>m</code>	Goto matching bracket (TS)	<code>match_brackets</code>
<code>s</code> <code><char></code>	Surround current selection with <code><char></code>	<code>surround_add</code>
<code>r</code> <code><from><to></code>	Replace surround character <code><from></code> with <code><to></code>	<code>surround_replace</code>
<code>d</code> <code><char></code>	Delete surround character <code><char></code>	<code>surround_delete</code>
<code>a</code> <code><object></code>	Select around textobject	<code>select_textobject_around</code>
<code>i</code> <code><object></code>	Select inside textobject	<code>select_textobject_inner</code>

TODO: Mappings for selecting syntax nodes (a superset of `[]`).

Window mode

Accessed by typing `Ctrl-w` in normal mode.

This layer is similar to Vim keybindings as Kakoune does not support windows.

Key	Description	Command
<code>w</code> , <code>Ctrl-w</code>	Switch to next window	<code>rotate_view</code>
<code>v</code> , <code>Ctrl-v</code>	Vertical right split	<code>vsplit</code>
<code>s</code> , <code>Ctrl-s</code>	Horizontal bottom split	<code>hsplit</code>
<code>f</code>	Go to files in the selections in horizontal splits	<code>goto_file</code>
<code>F</code>	Go to files in the selections in vertical splits	<code>goto_file</code>
<code>h</code> , <code>Ctrl-h</code> , Left	Move to left split	<code>jump_view_left</code>
<code>j</code> , <code>Ctrl-j</code> , Down	Move to split below	<code>jump_view_down</code>
<code>k</code> , <code>Ctrl-k</code> , Up	Move to split above	<code>jump_view_up</code>

Key	Description	Command
<code>l</code> , <code>Ctrl-l</code> , <code>Right</code>	Move to right split	<code>jump_view_right</code>
<code>q</code> , <code>Ctrl-q</code>	Close current window	<code>wclose</code>
<code>o</code> , <code>Ctrl-o</code>	Only keep the current window, closing all the others	<code>wonly</code>
<code>H</code>	Swap window to the left	<code>swap_view_left</code>
<code>J</code>	Swap window downwards	<code>swap_view_down</code>
<code>K</code>	Swap window upwards	<code>swap_view_up</code>
<code>L</code>	Swap window to the right	<code>swap_view_right</code>

Space mode

Accessed by typing `Space` in normal mode.

This layer is a kludge of mappings, mostly pickers.

Key	Description	Command
<code>f</code>	Open file picker at LSP workspace root	<code>file_picker</code>
<code>F</code>	Open file picker at current working directory	<code>file_picker_in_current_directory</code>
<code>b</code>	Open buffer picker	<code>buffer_picker</code>
<code>j</code>	Open jumplist picker	<code>jumplist_picker</code>
<code>g</code>	Open changed file picker	<code>changed_file_picker</code>
<code>G</code>	Debug (experimental)	N/A
<code>k</code>	Show documentation for item under cursor in a popup (LSP)	<code>hover</code>
<code>s</code>	Open document symbol picker (LSP)	<code>symbol_picker</code>

Key	Description	Command
S	Open workspace symbol picker (LSP)	<code>workspace_symbol_picker</code>
d	Open document diagnostics picker (LSP)	<code>diagnostics_picker</code>
D	Open workspace diagnostics picker (LSP)	<code>workspace_diagnostics_picker</code>
r	Rename symbol (LSP)	<code>rename_symbol</code>
a	Apply code action (LSP)	<code>code_action</code>
h	Select symbol references (LSP)	<code>select_references_to_symbol_under_cursor</code>
'	Open last fuzzy picker	<code>last_picker</code>
w	Enter window mode	N/A
c	Comment/uncomment selections	<code>toggle_comments</code>
C	Block comment/uncomment selections	<code>toggle_block_comments</code>
Alt -c	Line comment/uncomment selections	<code>toggle_line_comments</code>
p	Paste system clipboard after selections	<code>paste_clipboard_after</code>
P	Paste system clipboard before selections	<code>paste_clipboard_before</code>
y	Yank selections to clipboard	<code>yank_to_clipboard</code>
Y	Yank main selection to clipboard	<code>yank_main_selection_to_clipboard</code>
R	Replace selections by clipboard contents	<code>replace_selections_with_clipboard</code>
/	Global search in workspace folder	<code>global_search</code>
?	Open command palette	<code>command_palette</code>

💡 Global search displays results in a fuzzy picker, use `Space + '` to bring it back up after opening a file.

Popup

Displays documentation for item under cursor. Remapping currently not supported.

Key	Description
<code>Ctrl-u</code>	Scroll up
<code>Ctrl-d</code>	Scroll down

Completion Menu

Displays documentation for the selected completion item. Remapping currently not supported.

Key	Description
<code>Shift-Tab</code> , <code>Ctrl-p</code> , <code>Up</code>	Previous entry
<code>Tab</code> , <code>Ctrl-n</code> , <code>Down</code>	Next entry
<code>Enter</code>	Close menu and accept completion
<code>Ctrl-c</code>	Close menu and reject completion

Any other keypresses result in the completion being accepted.

Signature-help Popup

Displays the signature of the selected completion item. Remapping currently not supported.

Key	Description
<code>Alt-p</code>	Previous signature
<code>Alt-n</code>	Next signature

Unimpaired

These mappings are in the style of `vim-unimpaired`.

Key	Description	Command
<code>]d</code>	Go to next diagnostic (LSP)	<code>goto_next_diag</code>
<code>[d</code>	Go to previous diagnostic (LSP)	<code>goto_prev_diag</code>
<code>]D</code>	Go to last diagnostic in document (LSP)	<code>goto_last_diag</code>
<code>[D</code>	Go to first diagnostic in document (LSP)	<code>goto_first_diag</code>
<code>]f</code>	Go to next function (TS)	<code>goto_next_function</code>
<code>[f</code>	Go to previous function (TS)	<code>goto_prev_function</code>
<code>]t</code>	Go to next type definition (TS)	<code>goto_next_class</code>
<code>[t</code>	Go to previous type definition (TS)	<code>goto_prev_class</code>
<code>]a</code>	Go to next argument/parameter (TS)	<code>goto_next_parameter</code>
<code>[a</code>	Go to previous argument/parameter (TS)	<code>goto_prev_parameter</code>
<code>]c</code>	Go to next comment (TS)	<code>goto_next_comment</code>
<code>[c</code>	Go to previous comment (TS)	<code>goto_prev_comment</code>
<code>]T</code>	Go to next test (TS)	<code>goto_next_test</code>
<code>[T</code>	Go to previous test (TS)	<code>goto_prev_test</code>
<code>]p</code>	Go to next paragraph	<code>goto_next_paragraph</code>
<code>[p</code>	Go to previous paragraph	<code>goto_prev_paragraph</code>
<code>]g</code>	Go to next change	<code>goto_next_change</code>
<code>[g</code>	Go to previous change	<code>goto_prev_change</code>
<code>]G</code>	Go to last change	<code>goto_last_change</code>
<code>[G</code>	Go to first change	<code>goto_first_change</code>
<code>]Space</code>	Add newline below	<code>add_newline_below</code>
<code>[Space</code>	Add newline above	<code>add_newline_above</code>

Insert mode

Accessed by typing `i` in normal mode.

Insert mode bindings are minimal by default. Helix is designed to be a modal editor, and this is reflected in the user experience and internal mechanics. Changes to the text are only saved for undos when escaping from insert mode to normal mode.

 New users are strongly encouraged to learn the modal editing paradigm to get the smoothest experience.

Key	Description	Command
<code>Escape</code>	Switch to normal mode	<code>normal_mode</code>
<code>Ctrl-s</code>	Commit undo checkpoint	<code>commit_undo_checkpoint</code>
<code>Ctrl-x</code>	Autocomplete	<code>completion</code>
<code>Ctrl-r</code>	Insert a register content	<code>insert_register</code>
<code>Ctrl-w</code> , <code>Alt-Backspace</code>	Delete previous word	<code>delete_word_backward</code>
<code>Alt-d</code> , <code>Alt-Delete</code>	Delete next word	<code>delete_word_forward</code>
<code>Ctrl-u</code>	Delete to start of line	<code>kill_to_line_start</code>
<code>Ctrl-k</code>	Delete to end of line	<code>kill_to_line_end</code>
<code>Ctrl-h</code> , <code>Backspace</code> , <code>Shift-Backspace</code>	Delete previous char	<code>delete_char_backward</code>
<code>Ctrl-d</code> , <code>Delete</code>	Delete next char	<code>delete_char_forward</code>
<code>Ctrl-j</code> , <code>Enter</code>	Insert new line	<code>insert_newline</code>

These keys are not recommended, but are included for new users less familiar with modal editors.

Key	Description	Command
-----	-------------	---------

Key	Description	Command
Up	Move to previous line	move_line_up
Down	Move to next line	move_line_down
Left	Backward a char	move_char_left
Right	Forward a char	move_char_right
PageUp	Move one page up	page_up
PageDown	Move one page down	page_down
Home	Move to line start	goto_line_start
End	Move to line end	goto_line_end_newline

As you become more comfortable with modal editing, you may want to disable some insert mode bindings. You can do this by editing your `config.toml` file.

```
[keys.insert]
up = "no_op"
down = "no_op"
left = "no_op"
right = "no_op"
pageup = "no_op"
pagedown = "no_op"
home = "no_op"
end = "no_op"
```

Select / extend mode

Accessed by typing `v` in normal mode.

Select mode echoes Normal mode, but changes any movements to extend selections rather than replace them. Goto motions are also changed to extend, so that `vgl`, for example, extends the selection to the end of the line.

Search is also affected. By default, `n` and `N` will remove the current selection and select the next instance of the search term. Toggling this mode before pressing `n` or `N` makes it possible to keep the current selection. Toggling it on and off during your iterative searching

allows you to selectively add search terms to your selections.

Picker

Keys to use within picker. Remapping currently not supported. See the documentation page on [pickers](#) for more info. **Prompt** keybinds also work in pickers, except where they conflict with picker keybinds.

Key	Description
Shift-Tab , Up , Ctrl-p	Previous entry
Tab , Down , Ctrl-n	Next entry
PageUp , Ctrl-u	Page up
PageDown , Ctrl-d	Page down
Home	Go to first entry
End	Go to last entry
Enter	Open selected
Alt-Enter	Open selected in the background without closing the picker
Ctrl-s	Open horizontally
Ctrl-v	Open vertically
Ctrl-t	Toggle preview
Escape , Ctrl-c	Close picker

Prompt

Keys to use within prompt, Remapping currently not supported.

Key	Description
Escape , Ctrl-c	Close prompt

Key	Description
<code>Alt-b</code> , <code>Ctrl-Left</code>	Backward a word
<code>Ctrl-b</code> , <code>Left</code>	Backward a char
<code>Alt-f</code> , <code>Ctrl-Right</code>	Forward a word
<code>Ctrl-f</code> , <code>Right</code>	Forward a char
<code>Ctrl-e</code> , <code>End</code>	Move prompt end
<code>Ctrl-a</code> , <code>Home</code>	Move prompt start
<code>Ctrl-w</code> , <code>Alt-Backspace</code> , <code>Ctrl-Backspace</code>	Delete previous word
<code>Alt-d</code> , <code>Alt-Delete</code> , <code>Ctrl-Delete</code>	Delete next word
<code>Ctrl-u</code>	Delete to start of line
<code>Ctrl-k</code>	Delete to end of line
<code>Backspace</code> , <code>Ctrl-h</code> , <code>Shift-Backspace</code>	Delete previous char
<code>Delete</code> , <code>Ctrl-d</code>	Delete next char
<code>Ctrl-s</code>	Insert a word under doc cursor, may be changed to <code>Ctrl-r</code> <code>Ctrl-w</code> later
<code>Ctrl-p</code> , <code>Up</code>	Select previous history
<code>Ctrl-n</code> , <code>Down</code>	Select next history
<code>Ctrl-r</code>	Insert the content of the register selected by following input char
<code>Tab</code>	Select next completion item
<code>BackTab</code>	Select previous completion item
<code>Enter</code>	Open selected

Command line

- Quoting
- Flags
- Expansions
- Exceptions

The command line is used for executing typable commands like `:write` or `:quit`. Press `:` to activate the command line.

Typable commands optionally accept arguments. `:write` for example accepts an optional path to write the file contents. The command line also supports a quoting syntax for arguments, flags to modify command behaviors, and *expansions* - a way to insert values from the editor. Most commands support these features but some have custom parsing rules (see the *exceptions* below).

Quoting

By default, command arguments are split on tabs and space characters.

`:open README.md CHANGELOG.md` for example should open two files, `README.md` and `CHANGELOG.md`. Arguments that contain spaces can be surrounded in single quotes (`'`) or backticks (```) to prevent the space from separating the argument, like `:open 'a b.txt'`.

Double quotes may be used the same way, but double quotes *expand* their inner content. `:echo "%{cursor_line}"` for example may print `1` because of the expansion for the `cursor_line` variable. `:echo '%{cursor_line}'` though prints `%{cursor_line}` literally: content within single quotes or backticks is interpreted as-is.

On Unix systems the backslash character may be used to escape certain characters depending on where it is used. Within an argument which isn't surround in quotes, the backslash can be used to escape the space or tab characters: `:open a\ b.txt` is equivalent to `:open 'a b.txt'`. The backslash may also be used to escape quote characters (`'`, ```, `"`) or the percent token (`%`) when used at the beginning of an argument. `:echo \%sh{foo}` for example prints `%sh{foo}` instead of invoking a `foo` shell command and `:echo \"quote` prints `"quote`. The backslash character is treated literally in any other situation on Unix systems and always on

Windows: `:echo \n` always prints `\n`.

Flags

Command flags are optional switches that can be used to alter the behavior of a command. For example the `:sort` command accepts an optional `--reverse` (or `-r` for short) flag which causes the sort command to reverse the sorting direction. Typing the `-` character shows completions for the current command's flags, if any.

The `--` flag specifies the end of flags. All arguments after `--` are treated as positional arguments: `:open -- -a.txt` opens a file called `-a.txt`.

Expansions

Expansions are patterns that Helix recognizes and replaces within the command line. Helix recognizes anything starting with a percent token (`%`) as an expansion, for example `%sh{echo hi!}`. Expansions are particularly useful when used in commands like `:echo` or `:noop` for executing simple scripts. For example:

```
[keys.normal]
```

```
# Print the current line's git blame information to the statusline.  
space.B = ":echo %sh{git blame -L %s,+1 %s}"
```

Expansions take the form `%[<kind>]<open><contents><close>`. In `%sh{echo hi!}`, for example, the kind is `sh` - the shell expansion - and the contents are `"echo hi!"`, with `{` and `}` acting as opening and closing delimiters. The following open/close characters are recognized as expansion delimiter pairs: `(/)`, `[/]`, `{/}` and `</>`. Plus the single characters `'`, `"` or `|` may be used instead: `%{cursor_line}` is equivalent to `%<cursor_line>`, `%[cursor_line]` or `%|cursor_line|`.

To escape a percent character instead of treating it as an expansion, use two percent characters consecutively. To execute a shell command like `date -u +%Y-%m-%d'`, double the percent characters: `:echo %sh{date -u +%Y-%m-%d'}`.

When no `<kind>` is provided, Helix will expand a **variable**. For example

`%{cursor_line}` can be used as in argument to insert the line number.
`:echo %{cursor_line}` for instance may print `1` to the statusline.

The following variables are supported:

Name	Description
<code>cursor_line</code>	The line number of the primary cursor in the currently focused document, starting at 1.
<code>cursor_column</code>	The column number of the primary cursor in the currently focused document, starting at 1. This is counted as the number of grapheme clusters from the start of the line rather than bytes or codepoints.
<code>buffer_name</code>	The relative path of the currently focused document. <code>[scratch]</code> is expanded instead for scratch buffers.
<code>line_ending</code>	A string containing the line ending of the currently focused document. For example on Unix systems this is usually a line-feed character (<code>\n</code>) but on Windows systems this may be a carriage-return plus a line-feed (<code>\r\n</code>). The line ending kind of the currently focused document can be inspected with the <code>:line-ending</code> command.
<code>language</code>	A string containing the language name of the currently focused document.
<code>selection</code>	A string containing the contents of the primary selection of the currently focused document.
<code>selection_line_start</code>	The line number of the start of the primary selection in the currently focused document, starting at 1.
<code>selection_line_end</code>	The line number of the end of the primary selection in the currently focused document, starting at 1.

Aside from editor variables, the following expansions may be used:

- Unicode `%u{..}`. The contents may contain up to six hexadecimal numbers corresponding to a Unicode codepoint value. For example `:echo %u{25CF}` prints `☛` to the statusline.
- Shell `%sh{..}`. The contents are passed to the configured shell

command. For example `:echo %sh{echo "20 * 5" | bc}` may print `100` on the statusline on when using a shell with `echo` and the `bc` calculator installed. Shell expansions are evaluated recursively. `%sh{echo '%{buffer_name}:%{cursor_line}'}` for example executes a command like `echo 'README.md:1'`: the variables within the `%sh{..}` expansion are evaluated before executing the shell command.

As mentioned above, double quotes can be used to surround arguments containing spaces but also support expansions within the quoted content unlike single quotes or backticks. For example `:echo "circle: %u{25CF}"` prints `circle: ●` to the statusline while `:echo 'circle: %u{25CF}'` prints `circle: %u{25CF}`.

Note that expansions are only evaluated once the Enter key is pressed in command mode.

Exceptions

The following commands support expansions but otherwise pass the given argument directly to the shell program without interpreting quotes:

- `:insert-output`
- `:append-output`
- `:pipe`
- `:pipe-to`
- `:run-shell-command`

For example executing `:sh echo "%{buffer_name}:%{cursor_column}"` would pass text like `echo "README.md:1"` as an argument to the shell program: the expansions are evaluated but not the quotes. As mentioned above, percent characters can be used in shell commands by doubling the percent character. To insert the output of a command like `date -u +%Y-%m-%d` use `:insert-output date -u +%Y-%m-%d`.

The `:set-option` and `:toggle-option` commands use regular parsing for the first argument - the config option name - and parse the rest depending on the config option's type. `:set-option` interprets the second argument as a string for string config options and parses everything else as JSON.

`:toggle-option`'s behavior depends on the JSON type of the config option

supplied as the first argument:

- Booleans: only the config option name should be provided. For example `:toggle-option auto-format` will flip the `auto-format` option.
- Strings: the rest of the command line is parsed with regular quoting rules. For example `:toggle-option indent-heuristic hybrid tree-sitter simple` cycles through "hybrid", "tree-sitter" and "simple" values on each invocation of the command.
- Numbers, arrays and objects: the rest of the command line is parsed as a stream of JSON values. For example `:toggle-option rulers [81] [51, 73]` cycles through `[81]` and `[51, 73]`.

When providing multiple values to `:toggle-option` there should be no duplicates. `:toggle-option indent-heuristic hybrid simple tree-sitter simple` for example would only toggle between "hybrid" and "tree-sitter" values.

`:lsp-workspace-command` works similarly to `:toggle-option`. The first argument (if present) is parsed according to normal rules. The rest of the line is parsed as JSON values. Unlike `:toggle-option`, string arguments for a command must be quoted. For example `:lsp-workspace-command lsp.Command "foo" "bar"`.

Commands

- Typable commands
- Static commands

Typable commands

Typable commands are used from command mode and may take arguments. Command mode can be activated by pressing `:`. The built-in typable commands are:

Name	Description
<code>:quit</code> , <code>:q</code>	Close the current view.
<code>:quit!</code> , <code>:q!</code>	Force close the current view, ignoring unsaved changes.
<code>:open</code> , <code>:o</code> , <code>:edit</code> , <code>:e</code>	Open a file from disk into the current view.
<code>:buffer-close</code> , <code>:bc</code> , <code>:bclose</code>	Close the current buffer.
<code>:buffer-close!</code> , <code>:bc!</code> , <code>:bclose!</code>	Close the current buffer forcefully, ignoring unsaved changes.
<code>:buffer-close-others</code> , <code>:bco</code> , <code>:bcloseother</code>	Close all buffers but the currently focused one.
<code>:buffer-close-others!</code> , <code>:bco!</code> , <code>:bcloseother!</code>	Force close all buffers but the currently focused one.
<code>:buffer-close-all</code> , <code>:bca</code> , <code>:bcloseall</code>	Close all buffers without quitting.
<code>:buffer-close-all!</code> , <code>:bca!</code> , <code>:bcloseall!</code>	Force close all buffers ignoring unsaved changes without quitting.
<code>:buffer-next</code> , <code>:bn</code> , <code>:bnext</code>	Goto next buffer.

Name	Description
<code>:buffer-previous</code> , <code>:bp</code> , <code>:bprev</code>	Goto previous buffer.
<code>:write</code> , <code>:w</code>	Write changes to disk. Accepts an optional path (<code>:write some/path.txt</code>)
<code>:write!</code> , <code>:w!</code>	Force write changes to disk creating necessary subdirectories. Accepts an optional path (<code>:write! some/path.txt</code>)
<code>:write-buffer-close</code> , <code>:wbc</code>	Write changes to disk and closes the buffer. Accepts an optional path (<code>:write-buffer-close some/path.txt</code>)
<code>:write-buffer-close!</code> , <code>:wbc!</code>	Force write changes to disk creating necessary subdirectories and closes the buffer. Accepts an optional path (<code>:write-buffer-close! some/path.txt</code>)
<code>:new</code> , <code>:n</code>	Create a new scratch buffer.
<code>:format</code> , <code>:fmt</code>	Format the file using an external formatter or language server.
<code>:indent-style</code>	Set the indentation style for editing. ('t' for tabs or 1-16 for number of spaces.)
<code>:line-ending</code>	Set the document's default line ending. Options: <code>crlf</code> , <code>lf</code> .
<code>:earlier</code> , <code>:ear</code>	Jump back to an earlier point in edit history. Accepts a number of steps or a time span.
<code>:later</code> , <code>:lat</code>	Jump to a later point in edit history. Accepts a number of steps or a time span.
<code>:write-quit</code> , <code>:wq</code> , <code>:x</code>	Write changes to disk and close the current view. Accepts an optional path (<code>:wq some/path.txt</code>)
<code>:write-quit!</code> , <code>:wq!</code> , <code>:x!</code>	Write changes to disk and close the current view forcefully. Accepts an optional path (<code>:wq! some/path.txt</code>)
<code>:write-all</code> , <code>:wa</code>	Write changes from all buffers to disk.
<code>:write-all!</code> , <code>:wa!</code>	Forcefully write changes from all buffers to disk creating necessary subdirectories.

Name	Description
<code>:write-quit-all</code> , <code>:wqa</code> , <code>:xa</code>	Write changes from all buffers to disk and close all views.
<code>:write-quit-all!</code> , <code>:wqa!</code> , <code>:xa!</code>	Write changes from all buffers to disk and close all views forcefully (ignoring unsaved changes).
<code>:quit-all</code> , <code>:qa</code>	Close all views.
<code>:quit-all!</code> , <code>:qa!</code>	Force close all views ignoring unsaved changes.
<code>:cquit</code> , <code>:cq</code>	Quit with exit code (default 1). Accepts an optional integer exit code (<code>:cq 2</code>).
<code>:cquit!</code> , <code>:cq!</code>	Force quit with exit code (default 1) ignoring unsaved changes. Accepts an optional integer exit code (<code>:cq! 2</code>).
<code>:theme</code>	Change the editor theme (show current theme if no name specified).
<code>:yank-join</code>	Yank joined selections. A separator can be provided as first argument. Default value is newline.
<code>:clipboard-yank</code>	Yank main selection into system clipboard.
<code>:clipboard-yank-join</code>	Yank joined selections into system clipboard. A separator can be provided as first argument. Default value is newline.
<code>:primary-clipboard-yank</code>	Yank main selection into system primary clipboard.
<code>:primary-clipboard-yank-join</code>	Yank joined selections into system primary clipboard. A separator can be provided as first argument. Default value is newline.
<code>:clipboard-paste-after</code>	Paste system clipboard after selections.
<code>:clipboard-paste-before</code>	Paste system clipboard before selections.
<code>:clipboard-paste-replace</code>	Replace selections with content of system clipboard.
<code>:primary-clipboard-paste-after</code>	Paste primary clipboard after selections.

Name	Description
<code>:primary-clipboard-paste-before</code>	Paste primary clipboard before selections.
<code>:primary-clipboard-paste-replace</code>	Replace selections with content of system primary clipboard.
<code>:show-clipboard-provider</code>	Show clipboard provider name in status bar.
<code>:change-current-directory</code> , <code>:cd</code>	Change the current working directory.
<code>:show-directory</code> , <code>:pwd</code>	Show the current working directory.
<code>:encoding</code>	Set encoding. Based on https://encoding.spec.whatwg.org .
<code>:character-info</code> , <code>:char</code>	Get info about the character under the primary cursor.
<code>:reload</code> , <code>:rl</code>	Discard changes and reload from the source file.
<code>:reload-all</code> , <code>:rla</code>	Discard changes and reload all documents from the source files.
<code>:update</code> , <code>:u</code>	Write changes only if the file has been modified.
<code>:lsp-workspace-command</code>	Open workspace command picker
<code>:lsp-restart</code>	Restarts the given language servers, or all language servers that are used by the current file if no arguments are supplied
<code>:lsp-stop</code>	Stops the given language servers, or all language servers that are used by the current file if no arguments are supplied
<code>:tree-sitter-scopes</code>	Display tree sitter scopes, primarily for theming and development.
<code>:tree-sitter-highlight-name</code>	Display name of tree-sitter highlight scope under the cursor.
<code>:debug-start</code> , <code>:dbg</code>	Start a debug session from a given template with given parameters.

Name	Description
<code>:debug-remote</code> , <code>:dbg-tcp</code>	Connect to a debug adapter by TCP address and start a debugging session from a given template with given parameters.
<code>:debug-eval</code>	Evaluate expression in current debug context.
<code>:vsplit</code> , <code>:vs</code>	Open the file in a vertical split.
<code>:vsplit-new</code> , <code>:vnew</code>	Open a scratch buffer in a vertical split.
<code>:hsplit</code> , <code>:hs</code> , <code>:sp</code>	Open the file in a horizontal split.
<code>:hsplit-new</code> , <code>:hnew</code>	Open a scratch buffer in a horizontal split.
<code>:tutor</code>	Open the tutorial.
<code>:goto</code> , <code>:g</code>	Goto line number.
<code>:set-language</code> , <code>:lang</code>	Set the language of current buffer (show current language if no value specified).
<code>:set-option</code> , <code>:set</code>	Set a config option at runtime. For example to disable smart case search, use <code>:set search.smart-case false</code> .
<code>:toggle-option</code> , <code>:toggle</code>	Toggle a config option at runtime. For example to toggle smart case search, use <code>:toggle search.smart-case</code> .
<code>:get-option</code> , <code>:get</code>	Get the current value of a config option.
<code>:sort</code>	Sort ranges in selection.
<code>:reflow</code>	Hard-wrap the current selection of lines to a given width.
<code>:tree-sitter-subtree</code> , <code>:ts-subtree</code>	Display the smallest tree-sitter subtree that spans the primary selection, primarily for debugging queries.
<code>:config-reload</code>	Refresh user config.
<code>:config-open</code>	Open the user config.toml file.
<code>:config-open-workspace</code>	Open the workspace config.toml file.
<code>:log-open</code>	Open the helix log file.

Name	Description
<code>:insert-output</code>	Run shell command, inserting output before each selection.
<code>:append-output</code>	Run shell command, appending output after each selection.
<code>:pipe</code> , <code>: </code>	Pipe each selection to the shell command.
<code>:pipe-to</code>	Pipe each selection to the shell command, ignoring output.
<code>:run-shell-command</code> , <code>:sh</code> , <code>:!</code>	Run a shell command
<code>:reset-diff-change</code> , <code>:diffget</code> , <code>:diffg</code>	Reset the diff change at the cursor position.
<code>:clear-register</code>	Clear given register. If no argument is provided, clear all registers.
<code>:redraw</code>	Clear and re-render the whole UI
<code>:move</code> , <code>:mv</code>	Move the current buffer and its corresponding file to a different path
<code>:yank-diagnostic</code>	Yank diagnostic(s) under primary cursor to register, or clipboard by default
<code>:read</code> , <code>:r</code>	Load a file into buffer
<code>:echo</code>	Prints the given arguments to the statusline.
<code>:noop</code>	Does nothing.

Static Commands

Static commands take no arguments and can be bound to keys. Static commands can also be executed from the command picker (`<space>?`). The built-in static commands are:

Name	Description	Default keybinds
<code>no_op</code>	Do nothing	

Name	Description	Default keybinds
<code>move_char_left</code>	Move left	normal: <code>h</code> , <code><left></code> , insert: <code><left></code>
<code>move_char_right</code>	Move right	normal: <code>l</code> , <code><right></code> , insert: <code><right></code>
<code>move_line_up</code>	Move up	normal: <code>gk</code>
<code>move_line_down</code>	Move down	normal: <code>gj</code>
<code>move_visual_line_up</code>	Move up	normal: <code>k</code> , <code><up></code> , insert: <code><up></code>
<code>move_visual_line_down</code>	Move down	normal: <code>j</code> , <code><down></code> , insert: <code><down></code>
<code>extend_char_left</code>	Extend left	select: <code>h</code> , <code><left></code>
<code>extend_char_right</code>	Extend right	select: <code>l</code> , <code><right></code>
<code>extend_line_up</code>	Extend up	select: <code>gk</code>
<code>extend_line_down</code>	Extend down	select: <code>gj</code>
<code>extend_visual_line_up</code>	Extend up	select: <code>k</code> , <code><up></code>
<code>extend_visual_line_down</code>	Extend down	select: <code>j</code> , <code><down></code>
<code>copy_selection_on_next_line</code>	Copy selection on next line	normal: <code>C</code> , select: <code>C</code>
<code>copy_selection_on_prev_line</code>	Copy selection on previous line	normal: <code><A-C></code> , select: <code><A-C></code>
<code>move_next_word_start</code>	Move to start of next word	normal: <code>w</code>

Name	Description	Default keybinds
<code>move_prev_word_start</code>	Move to start of previous word	normal: <code>b</code>
<code>move_next_word_end</code>	Move to end of next word	normal: <code>e</code>
<code>move_prev_word_end</code>	Move to end of previous word	
<code>move_next_long_word_start</code>	Move to start of next long word	normal: <code>W</code>
<code>move_prev_long_word_start</code>	Move to start of previous long word	normal: <code>B</code>
<code>move_next_long_word_end</code>	Move to end of next long word	normal: <code>E</code>
<code>move_prev_long_word_end</code>	Move to end of previous long word	
<code>move_next_sub_word_start</code>	Move to start of next sub word	
<code>move_prev_sub_word_start</code>	Move to start of previous sub word	
<code>move_next_sub_word_end</code>	Move to end of next sub word	
<code>move_prev_sub_word_end</code>	Move to end of previous sub word	
<code>move_parent_node_end</code>	Move to end of the parent node	normal: <code><A-e></code>
<code>move_parent_node_start</code>	Move to beginning of the parent	normal: <code><A-b></code>

Name	Description	Default keybinds
	node	
<code>extend_next_word_start</code>	Extend to start of next word	select: <code>w</code>
<code>extend_prev_word_start</code>	Extend to start of previous word	select: <code>b</code>
<code>extend_next_word_end</code>	Extend to end of next word	select: <code>e</code>
<code>extend_prev_word_end</code>	Extend to end of previous word	
<code>extend_next_long_word_start</code>	Extend to start of next long word	select: <code>W</code>
<code>extend_prev_long_word_start</code>	Extend to start of previous long word	select: <code>B</code>
<code>extend_next_long_word_end</code>	Extend to end of next long word	select: <code>E</code>
<code>extend_prev_long_word_end</code>	Extend to end of prev long word	
<code>extend_next_sub_word_start</code>	Extend to start of next sub word	
<code>extend_prev_sub_word_start</code>	Extend to start of previous sub word	
<code>extend_next_sub_word_end</code>	Extend to end of next sub word	

Name	Description	Default keybinds
<code>extend_prev_sub_word_end</code>	Extend to end of prev sub word	
<code>extend_parent_node_end</code>	Extend to end of the parent node	select: <code><A-e></code>
<code>extend_parent_node_start</code>	Extend to beginning of the parent node	select: <code><A-b></code>
<code>find_till_char</code>	Move till next occurrence of char	normal: <code>t</code>
<code>find_next_char</code>	Move to next occurrence of char	normal: <code>f</code>
<code>extend_till_char</code>	Extend till next occurrence of char	select: <code>t</code>
<code>extend_next_char</code>	Extend to next occurrence of char	select: <code>f</code>
<code>till_prev_char</code>	Move till previous occurrence of char	normal: <code>T</code>
<code>find_prev_char</code>	Move to previous occurrence of char	normal: <code>F</code>
<code>extend_till_prev_char</code>	Extend till previous occurrence of char	select: <code>T</code>

Name	Description	Default keybinds
<code>extend_prev_char</code>	Extend to previous occurrence of char	select: <code>F</code>
<code>repeat_last_motion</code>	Repeat last motion	normal: <code><A-.></code> , select: <code><A-.></code>
<code>replace</code>	Replace with new char	normal: <code>r</code> , select: <code>r</code>
<code>switch_case</code>	Switch (toggle) case	normal: <code>~</code> , select: <code>~</code>
<code>switch_to_uppercase</code>	Switch to uppercase	normal: <code><A-`></code> , select: <code><A-`></code>
<code>switch_to_lowercase</code>	Switch to lowercase	normal: <code>`</code> , select: <code>`</code>
<code>page_up</code>	Move page up	normal: <code><C-b></code> , <code>Z<C-b></code> , <code>z<C-b></code> , <code><pageup></code> , <code>Z<pageup></code> , <code>z<pageup></code> , select: <code><C-b></code> , <code>Z<C-b></code> , <code>z<C-b></code> , <code><pageup></code> , <code>Z<pageup></code> , <code>z<pageup></code> , insert: <code><pageup></code>
<code>page_down</code>	Move page down	normal: <code><C-f></code> , <code>Z<C-f></code> , <code>z<C-f></code> , <code><pagedown></code> , <code>Z<pagedown></code> , <code>z<pagedown></code> , select: <code><C-f></code> , <code>Z<C-f></code> , <code>z<C-f></code> , <code><pagedown></code> , <code>Z<pagedown></code> , <code>z<pagedown></code> , insert: <code><pagedown></code>

Name	Description	Default keybinds
<code>half_page_up</code>	Move half page up	
<code>half_page_down</code>	Move half page down	
<code>page_cursor_up</code>	Move page and cursor up	
<code>page_cursor_down</code>	Move page and cursor down	
<code>page_cursor_half_up</code>	Move page and cursor half up	normal: <code><C-u></code> , <code>Z<C-u></code> , <code>z<C-u></code> , <code>Z<backspace></code> , <code>z<backspace></code> , select: <code><C-u></code> , <code>Z<C-u></code> , <code>z<C-u></code> , <code>Z<backspace></code> , <code>z<backspace></code>
<code>page_cursor_half_down</code>	Move page and cursor half down	normal: <code><C-d></code> , <code>Z<C-d></code> , <code>z<C-d></code> , <code>Z<space></code> , <code>z<space></code> , select: <code><C-d></code> , <code>Z<C-d></code> , <code>z<C-d></code> , <code>Z<space></code> , <code>z<space></code>
<code>select_all</code>	Select whole document	normal: <code>%</code> , select: <code>%</code>
<code>select_regex</code>	Select all regex matches inside selections	normal: <code>s</code> , select: <code>s</code>
<code>split_selection</code>	Split selections on regex matches	normal: <code>S</code> , select: <code>S</code>
<code>split_selection_on_newline</code>	Split selection on newlines	normal: <code><A-s></code> , select: <code><A-s></code>

Name	Description	Default keybinds
<code>merge_selections</code>	Merge selections	normal: <code><A-minus></code> , select: <code><A-minus></code>
<code>merge_consecutive_selections</code>	Merge consecutive selections	normal: <code><A-_></code> , select: <code><A-_></code>
<code>search</code>	Search for regex pattern	normal: <code>/</code> , <code>Z/</code> , <code>z/</code> , select: <code>/</code> , <code>Z/</code> , <code>z/</code>
<code>rsearch</code>	Reverse search for regex pattern	normal: <code>?</code> , <code>Z?</code> , <code>z?</code> , select: <code>?</code> , <code>Z?</code> , <code>z?</code>
<code>search_next</code>	Select next search match	normal: <code>n</code> , <code>Zn</code> , <code>zn</code> , select: <code>Zn</code> , <code>zn</code>
<code>search_prev</code>	Select previous search match	normal: <code>N</code> , <code>ZN</code> , <code>zN</code> , select: <code>ZN</code> , <code>zN</code>
<code>extend_search_next</code>	Add next search match to selection	select: <code>n</code>
<code>extend_search_prev</code>	Add previous search match to selection	select: <code>N</code>
<code>search_selection</code>	Use current selection as search pattern	normal: <code><A-*></code> , select: <code><A-*></code>
<code>search_selection_detect_word_boundaries</code>	Use current selection as the search pattern, automatically wrapping with <code>\b</code> on word boundaries	normal: <code>*</code> , select: <code>*</code>

Name	Description	Default keybinds
<code>make_search_word_bounded</code>	Modify current search to make it word bounded	
<code>global_search</code>	Global search in workspace folder	normal: <code><space>/</code> , select: <code><space>/</code>
<code>extend_line</code>	Select current line, if already selected, extend to another line based on the anchor	
<code>extend_line_below</code>	Select current line, if already selected, extend to next line	normal: <code>x</code> , select: <code>x</code>
<code>extend_line_above</code>	Select current line, if already selected, extend to previous line	
<code>select_line_above</code>	Select current line, if already selected, extend or shrink line above based on the anchor	
<code>select_line_below</code>	Select current line, if already	

Name	Description	Default keybinds
	selected, extend or shrink line below based on the anchor	
<code>extend_to_line_bounds</code>	Extend selection to line bounds	normal: <code>x</code> , select: <code>x</code>
<code>shrink_to_line_bounds</code>	Shrink selection to line bounds	normal: <code><A-x></code> , select: <code><A-x></code>
<code>delete_selection</code>	Delete selection	normal: <code>d</code> , select: <code>d</code>
<code>delete_selection_noyank</code>	Delete selection without yanking	normal: <code><A-d></code> , select: <code><A-d></code>
<code>change_selection</code>	Change selection	normal: <code>c</code> , select: <code>c</code>
<code>change_selection_noyank</code>	Change selection without yanking	normal: <code><A-c></code> , select: <code><A-c></code>
<code>collapse_selection</code>	Collapse selection into single cursor	normal: <code>;</code> , select: <code>;</code>
<code>flip_selections</code>	Flip selection cursor and anchor	normal: <code><A-;></code> , select: <code><A-;></code>
<code>ensure_selections_forward</code>	Ensure all selections face forward	normal: <code><A-:></code> , select: <code><A-:></code>
<code>insert_mode</code>	Insert before selection	normal: <code>i</code> , select: <code>i</code>

Name	Description	Default keybinds
<code>append_mode</code>	Append after selection	normal: <code>a</code> , select: <code>a</code>
<code>command_mode</code>	Enter command mode	normal: <code>:</code> , select: <code>:</code>
<code>file_picker</code>	Open file picker	normal: <code><space>f</code> , select: <code><space>f</code>
<code>file_picker_in_current_buffer_directory</code>	Open file picker at current buffer's directory	
<code>file_picker_in_current_directory</code>	Open file picker at current working directory	normal: <code><space>F</code> , select: <code><space>F</code>
<code>file_explorer</code>	Open file explorer in workspace root	normal: <code><space>e</code> , select: <code><space>e</code>
<code>file_explorer_in_current_buffer_directory</code>	Open file explorer at current buffer's directory	normal: <code><space>E</code> , select: <code><space>E</code>
<code>file_explorer_in_current_directory</code>	Open file explorer at current working directory	
<code>code_action</code>	Perform code action	normal: <code><space>a</code> , select: <code><space>a</code>

Name	Description	Default keybinds
<code>buffer_picker</code>	Open buffer picker	normal: <code><space>b</code> , select: <code><space>b</code>
<code>jumplist_picker</code>	Open jumplist picker	normal: <code><space>j</code> , select: <code><space>j</code>
<code>symbol_picker</code>	Open symbol picker	normal: <code><space>s</code> , select: <code><space>s</code>
<code>changed_file_picker</code>	Open changed file picker	normal: <code><space>g</code> , select: <code><space>g</code>
<code>select_references_to_symbol_under_cursor</code>	Select symbol references	normal: <code><space>h</code> , select: <code><space>h</code>
<code>workspace_symbol_picker</code>	Open workspace symbol picker	normal: <code><space>S</code> , select: <code><space>S</code>
<code>diagnostics_picker</code>	Open diagnostic picker	normal: <code><space>d</code> , select: <code><space>d</code>
<code>workspace_diagnostics_picker</code>	Open workspace diagnostic picker	normal: <code><space>D</code> , select: <code><space>D</code>
<code>last_picker</code>	Open last picker	normal: <code><space>'</code> , select: <code><space>'</code>

Name	Description	Default keybinds
<code>insert_at_line_start</code>	Insert at start of line	normal: <code>I</code> , select: <code>I</code>
<code>insert_at_line_end</code>	Insert at end of line	normal: <code>A</code> , select: <code>A</code>
<code>open_below</code>	Open new line below selection	normal: <code>o</code> , select: <code>o</code>
<code>open_above</code>	Open new line above selection	normal: <code>O</code> , select: <code>O</code>
<code>normal_mode</code>	Enter normal mode	normal: <code><esc></code> , select: <code>v</code> , insert: <code><esc></code>
<code>select_mode</code>	Enter selection extend mode	normal: <code>v</code>
<code>exit_select_mode</code>	Exit selection mode	select: <code><esc></code>
<code>goto_definition</code>	Goto definition	normal: <code>gd</code> , select: <code>gd</code>
<code>goto_declaration</code>	Goto declaration	normal: <code>gD</code> , select: <code>gD</code>
<code>add_newline_above</code>	Add newline above	normal: <code>[<space></code> , select: <code>[<space></code>
<code>add_newline_below</code>	Add newline below	normal: <code>]<space></code> , select: <code>]<space></code>
<code>goto_type_definition</code>	Goto type definition	normal: <code>gy</code> , select: <code>gy</code>
<code>goto_implementation</code>	Goto implementation	normal: <code>gi</code> , select: <code>gi</code>

Name	Description	Default keybinds
<code>goto_file_start</code>	Goto line number else file start	normal: <code>gg</code>
<code>goto_file_end</code>	Goto file end	
<code>extend_to_file_start</code>	Extend to line number else file start	select: <code>gg</code>
<code>extend_to_file_end</code>	Extend to file end	
<code>goto_file</code>	Goto files/URLs in selections	normal: <code>gf</code> , select: <code>gf</code>
<code>goto_file_hsplit</code>	Goto files in selections (hsplit)	normal: <code><C-w>f</code> , <code><space>wf</code> , select: <code><C-w>f</code> , <code><space>wf</code>
<code>goto_file_vsplit</code>	Goto files in selections (vsplit)	normal: <code><C-w>F</code> , <code><space>wF</code> , select: <code><C-w>F</code> , <code><space>wF</code>
<code>goto_reference</code>	Goto references	normal: <code>gr</code> , select: <code>gr</code>
<code>goto_window_top</code>	Goto window top	normal: <code>gt</code> , select: <code>gt</code>
<code>goto_window_center</code>	Goto window center	normal: <code>gc</code> , select: <code>gc</code>
<code>goto_window_bottom</code>	Goto window bottom	normal: <code>gb</code> , select: <code>gb</code>
<code>goto_last_accessed_file</code>	Goto last accessed file	normal: <code>ga</code> , select: <code>ga</code>
<code>goto_last_modified_file</code>	Goto last modified file	normal: <code>gm</code> , select: <code>gm</code>

Name	Description	Default keybinds
<code>goto_last_modification</code>	Goto last modification	normal: <code>g.</code> , select: <code>g.</code>
<code>goto_line</code>	Goto line	normal: <code>G</code> , select: <code>G</code>
<code>goto_last_line</code>	Goto last line	normal: <code>ge</code>
<code>extend_to_last_line</code>	Extend to last line	select: <code>ge</code>
<code>goto_first_diag</code>	Goto first diagnostic	normal: <code>[D</code> , select: <code>[D</code>
<code>goto_last_diag</code>	Goto last diagnostic	normal: <code>]D</code> , select: <code>]D</code>
<code>goto_next_diag</code>	Goto next diagnostic	normal: <code>]d</code> , select: <code>]d</code>
<code>goto_prev_diag</code>	Goto previous diagnostic	normal: <code>[d</code> , select: <code>[d</code>
<code>goto_next_change</code>	Goto next change	normal: <code>]g</code> , select: <code>]g</code>
<code>goto_prev_change</code>	Goto previous change	normal: <code>[g</code> , select: <code>[g</code>
<code>goto_first_change</code>	Goto first change	normal: <code>[G</code> , select: <code>[G</code>
<code>goto_last_change</code>	Goto last change	normal: <code>]G</code> , select: <code>]G</code>
<code>goto_line_start</code>	Goto line start	normal: <code>gh</code> , <code><home></code> , select: <code>gh</code> , insert: <code><home></code>
<code>goto_line_end</code>	Goto line end	normal: <code>gl</code> , <code><end></code> , select: <code>gl</code>
<code>goto_column</code>	Goto column	normal: <code>g </code>

Name	Description	Default keybinds
<code>extend_to_column</code>	Extend to column	select: <code>g </code>
<code>goto_next_buffer</code>	Goto next buffer	normal: <code>gn</code> , select: <code>gn</code>
<code>goto_previous_buffer</code>	Goto previous buffer	normal: <code>gp</code> , select: <code>gp</code>
<code>goto_line_end_newline</code>	Goto newline at line end	insert: <code><end></code>
<code>goto_first_nonwhitespace</code>	Goto first non-blank in line	normal: <code>gs</code> , select: <code>gs</code>
<code>trim_selections</code>	Trim whitespace from selections	normal: <code>-</code> , select: <code>-</code>
<code>extend_to_line_start</code>	Extend to line start	select: <code><home></code>
<code>extend_to_first_nonwhitespace</code>	Extend to first non-blank in line	
<code>extend_to_line_end</code>	Extend to line end	select: <code><end></code>
<code>extend_to_line_end_newline</code>	Extend to line end	
<code>signature_help</code>	Show signature help	
<code>smart_tab</code>	Insert tab if all cursors have all whitespace to their left; otherwise, run a separate command.	insert: <code><tab></code>

Name	Description	Default keybinds
<code>insert_tab</code>	Insert tab char	insert: <code><S-tab></code>
<code>insert_newline</code>	Insert newline char	insert: <code><C-j></code> , <code><ret></code>
<code>delete_char_backward</code>	Delete previous char	insert: <code><C-h></code> , <code><backspace></code> , <code><S-backspace></code>
<code>delete_char_forward</code>	Delete next char	insert: <code><C-d></code> , <code></code>
<code>delete_word_backward</code>	Delete previous word	insert: <code><C-w></code> , <code><A-backspace></code>
<code>delete_word_forward</code>	Delete next word	insert: <code><A-d></code> , <code><A-del></code>
<code>kill_to_line_start</code>	Delete till start of line	insert: <code><C-u></code>
<code>kill_to_line_end</code>	Delete till end of line	insert: <code><C-k></code>
<code>undo</code>	Undo change	normal: <code>u</code> , select: <code>u</code>
<code>redo</code>	Redo change	normal: <code>U</code> , select: <code>U</code>
<code>earlier</code>	Move backward in history	normal: <code><A-u></code> , select: <code><A-u></code>
<code>later</code>	Move forward in history	normal: <code><A-U></code> , select: <code><A-U></code>
<code>commit_undo_checkpoint</code>	Commit changes to new checkpoint	insert: <code><C-s></code>
<code>yank</code>	Yank selection	normal: <code>y</code> , select: <code>y</code>
<code>yank_to_clipboard</code>	Yank selections to clipboard	normal: <code><space>y</code> , select: <code><space>y</code>

Name	Description	Default keybinds
		<code><space>y</code>
<code>yank_to_primary_clipboard</code>	Yank selections to primary clipboard	
<code>yank_joined</code>	Join and yank selections	
<code>yank_joined_to_clipboard</code>	Join and yank selections to clipboard	
<code>yank_main_selection_to_clipboard</code>	Yank main selection to clipboard	normal: <code><space>Y</code> , select: <code><space>Y</code>
<code>yank_joined_to_primary_clipboard</code>	Join and yank selections to primary clipboard	
<code>yank_main_selection_to_primary_clipboard</code>	Yank main selection to primary clipboard	
<code>replace_with_yanked</code>	Replace with yanked text	normal: <code>R</code> , select: <code>R</code>
<code>replace_selections_with_clipboard</code>	Replace selections by clipboard content	normal: <code><space>R</code> , select: <code><space>R</code>
<code>replace_selections_with_primary_clipboard</code>	Replace selections by primary clipboard	
<code>paste_after</code>	Paste after selection	normal: <code>p</code> , select: <code>p</code>

Name	Description	Default keybinds
<code>paste_before</code>	Paste before selection	normal: <code>P</code> , select: <code>P</code>
<code>paste_clipboard_after</code>	Paste clipboard after selections	normal: <code><space>p</code> , select: <code><space>p</code>
<code>paste_clipboard_before</code>	Paste clipboard before selections	normal: <code><space>P</code> , select: <code><space>P</code>
<code>paste_primary_clipboard_after</code>	Paste primary clipboard after selections	
<code>paste_primary_clipboard_before</code>	Paste primary clipboard before selections	
<code>indent</code>	Indent selection	normal: <code><gt></code> , select: <code><gt></code>
<code>unindent</code>	Unindent selection	normal: <code><lt></code> , select: <code><lt></code>
<code>format_selections</code>	Format selection	normal: <code>=</code> , select: <code>=</code>
<code>join_selections</code>	Join lines inside selection	normal: <code>J</code> , select: <code>J</code>
<code>join_selections_space</code>	Join lines inside selection and select spaces	normal: <code><A-J></code> , select: <code><A-J></code>
<code>keep_selections</code>	Keep selections matching regex	normal: <code>K</code> , select: <code>K</code>

Name	Description	Default keybinds
<code>remove_selections</code>	Remove selections matching regex	normal: <code><A-K></code> , select: <code><A-K></code>
<code>align_selections</code>	Align selections in column	normal: <code>&</code> , select: <code>&</code>
<code>keep_primary_selection</code>	Keep primary selection	normal: <code>,</code> , select: <code>,</code>
<code>remove_primary_selection</code>	Remove primary selection	normal: <code><A-,></code> , select: <code><A-,></code>
<code>completion</code>	Invoke completion popup	insert: <code><C-x></code>
<code>hover</code>	Show docs for item under cursor	normal: <code><space>k</code> , select: <code><space>k</code>
<code>toggle_comments</code>	Comment/uncomment selections	normal: <code><C-c></code> , <code><space>c</code> , select: <code><C-c></code> , <code><space>c</code>
<code>toggle_line_comments</code>	Line comment/uncomment selections	normal: <code><space><A-c></code> , select: <code><space><A-c></code>
<code>toggle_block_comments</code>	Block comment/uncomment selections	normal: <code><space>C</code> , select: <code><space>C</code>
<code>rotate_selections_forward</code>	Rotate selections forward	normal: <code>)</code> , select: <code>)</code>
<code>rotate_selections_backward</code>	Rotate selections backward	normal: <code>(</code> , select: <code>(</code>

Name	Description	Default keybinds
<code>rotate_selection_contents_forward</code>	Rotate selection contents forward	normal: <code><A-></code> , select: <code><A-></code>
<code>rotate_selection_contents_backward</code>	Rotate selections contents backward	normal: <code><A-(</code> , select: <code><A-(</code>
<code>reverse_selection_contents</code>	Reverse selections contents	
<code>expand_selection</code>	Expand selection to parent syntax node	normal: <code><A-o</code> , <code><A-up</code> , select: <code><A-o</code> , <code><A-up</code>
<code>shrink_selection</code>	Shrink selection to previously expanded syntax node	normal: <code><A-i</code> , <code><A-down</code> , select: <code><A-i</code> , <code><A-down</code>
<code>select_next_sibling</code>	Select next sibling in the syntax tree	normal: <code><A-n</code> , <code><A-right</code> , select: <code><A-n</code> , <code><A-right</code>
<code>select_prev_sibling</code>	Select previous sibling the in syntax tree	normal: <code><A-p</code> , <code><A-left</code> , select: <code><A-p</code> , <code><A-left</code>
<code>select_all_siblings</code>	Select all siblings of the current node	normal: <code><A-a</code> , select: <code><A-a</code>
<code>select_all_children</code>	Select all children of the current node	normal: <code><A-I</code> , <code><S-A-down</code> , select: <code><A-I</code> , <code><S-A-down</code>

Name	Description	Default keybinds
<code>jump_forward</code>	Jump forward on jumplist	normal: <code><C-i></code> , <code><tab></code> , select: <code><C-i></code> , <code><tab></code>
<code>jump_backward</code>	Jump backward on jumplist	normal: <code><C-o></code> , select: <code><C-o></code>
<code>save_selection</code>	Save current selection to jumplist	normal: <code><C-s></code> , select: <code><C-s></code>
<code>jump_view_right</code>	Jump to right split	normal: <code><C-w>l</code> , <code><space>wl</code> , <code><C-w><C-l></code> , <code><C-w><right></code> , <code><space>w<C-l></code> , <code><space>w<right></code> , select: <code><C-w>l</code> , <code><space>wl</code> , <code><C-w><C-l></code> , <code><C-w><right></code> , <code><space>w<C-l></code> , <code><space>w<right></code>
<code>jump_view_left</code>	Jump to left split	normal: <code><C-w>h</code> , <code><space>wh</code> , <code><C-w><C-h></code> , <code><C-w><left></code> , <code><space>w<C-h></code> , <code><space>w<left></code> , select: <code><C-w>h</code> , <code><space>wh</code> , <code><C-w><C-h></code> , <code><C-w><left></code> , <code><space>w<C-h></code> , <code><space>w<left></code>
<code>jump_view_up</code>	Jump to split above	normal: <code><C-w>k</code> , <code><C-w><up></code> , <code><space>wk</code> , <code><C-w><C-k></code> , <code><space>w<up></code> , <code><space>w<C-k></code> , select: <code><C-w>k</code> , <code><C-w><up></code> ,

Name	Description	Default keybinds
		<code><space>wk</code> , <code><C-w><C-k></code> , <code><space>w<up></code> , <code><space>w<C-k></code>
<code>jump_view_down</code>	Jump to split below	normal: <code><C-w>j</code> , <code><space>wj</code> , <code><C-w><C-j></code> , <code><C-w><down></code> , <code><space>w<C-j></code> , <code><space>w<down></code> , select: <code><C-w>j</code> , <code><space>wj</code> , <code><C-w><C-j></code> , <code><C-w><down></code> , <code><space>w<C-j></code> , <code><space>w<down></code>
<code>swap_view_right</code>	Swap with right split	normal: <code><C-w>L</code> , <code><space>wL</code> , select: <code><C-w>L</code> , <code><space>wL</code>
<code>swap_view_left</code>	Swap with left split	normal: <code><C-w>H</code> , <code><space>wH</code> , select: <code><C-w>H</code> , <code><space>wH</code>
<code>swap_view_up</code>	Swap with split above	normal: <code><C-w>K</code> , <code><space>wK</code> , select: <code><C-w>K</code> , <code><space>wK</code>
<code>swap_view_down</code>	Swap with split below	normal: <code><C-w>J</code> , <code><space>wJ</code> , select: <code><C-w>J</code> , <code><space>wJ</code>

Name	Description	Default keybinds
<code>transpose_view</code>	Transpose splits	normal: <code><C-w>t</code> , <code><space>wt</code> , <code><C-w><C-t></code> , <code><space>w<C-t></code> , select: <code><C-w>t</code> , <code><space>wt</code> , <code><C-w><C-t></code> , <code><space>w<C-t></code>
<code>rotate_view</code>	Goto next window	normal: <code><C-w>w</code> , <code><space>ww</code> , <code><C-w><C-w></code> , <code><space>w<C-w></code> , select: <code><C-w>w</code> , <code><space>ww</code> , <code><C-w><C-w></code> , <code><space>w<C-w></code>
<code>rotate_view_reverse</code>	Goto previous window	
<code>hsplit</code>	Horizontal bottom split	normal: <code><C-w>s</code> , <code><space>ws</code> , <code><C-w><C-s></code> , <code><space>w<C-s></code> , select: <code><C-w>s</code> , <code><space>ws</code> , <code><C-w><C-s></code> , <code><space>w<C-s></code>
<code>hsplit_new</code>	Horizontal bottom split scratch buffer	normal: <code><C-w>ns</code> , <code><space>wns</code> , <code><C-w>n<C-s></code> , <code><space>wn<C-s></code> , select: <code><C-w>ns</code> , <code><space>wns</code> , <code><C-w>n<C-s></code> , <code><space>wn<C-s></code>
<code>vsplit</code>	Vertical right split	normal: <code><C-w>v</code> , <code><space>wv</code> , <code><C-w><C-v></code> , <code><space>w<C-v></code> ,

Name	Description	Default keybinds
		select: <code><C-w>v</code> , <code><space>wv</code> , <code><C-w><C-v></code> , <code><space>w<C-v></code>
<code>vsplit_new</code>	Vertical right split scratch buffer	normal: <code><C-w>nv</code> , <code><space>wnv</code> , <code><C-w>n<C-v></code> , <code><space>wn<C-v></code> , select: <code><C-w>nv</code> , <code><space>wnv</code> , <code><C-w>n<C-v></code> , <code><space>wn<C-v></code>
<code>wclose</code>	Close window	normal: <code><C-w>q</code> , <code><space>wq</code> , <code><C-w><C-q></code> , <code><space>w<C-q></code> , select: <code><C-w>q</code> , <code><space>wq</code> , <code><C-w><C-q></code> , <code><space>w<C-q></code>
<code>wonly</code>	Close windows except current	normal: <code><C-w>o</code> , <code><space>wo</code> , <code><C-w><C-o></code> , <code><space>w<C-o></code> , select: <code><C-w>o</code> , <code><space>wo</code> , <code><C-w><C-o></code> , <code><space>w<C-o></code>
<code>select_register</code>	Select register	normal: <code>"</code> , select: <code>"</code>
<code>insert_register</code>	Insert register	insert: <code><C-r></code>
<code>copy_between_registers</code>	Copy between two registers	

Name	Description	Default keybinds
<code>align_view_middle</code>	Align view middle	normal: <code>Zm</code> , <code>zm</code> , select: <code>Zm</code> , <code>zm</code>
<code>align_view_top</code>	Align view top	normal: <code>Zt</code> , <code>zt</code> , select: <code>Zt</code> , <code>zt</code>
<code>align_view_center</code>	Align view center	normal: <code>Zc</code> , <code>Zz</code> , <code>zc</code> , <code>zz</code> , select: <code>Zc</code> , <code>Zz</code> , <code>zc</code> , <code>zz</code>
<code>align_view_bottom</code>	Align view bottom	normal: <code>Zb</code> , <code>zb</code> , select: <code>Zb</code> , <code>zb</code>
<code>scroll_up</code>	Scroll view up	normal: <code>Zk</code> , <code>zk</code> , <code>Z<up></code> , <code>z<up></code> , select: <code>Zk</code> , <code>zk</code> , <code>Z<up></code> , <code>z<up></code>
<code>scroll_down</code>	Scroll view down	normal: <code>Zj</code> , <code>zj</code> , <code>Z<down></code> , <code>z<down></code> , select: <code>Zj</code> , <code>zj</code> , <code>Z<down></code> , <code>z<down></code>
<code>match_brackets</code>	Goto matching bracket	normal: <code>mm</code> , select: <code>mm</code>
<code>surround_add</code>	Surround add	normal: <code>ms</code> , select: <code>ms</code>
<code>surround_replace</code>	Surround replace	normal: <code>mr</code> , select: <code>mr</code>
<code>surround_delete</code>	Surround delete	normal: <code>md</code> , select: <code>md</code>
<code>select_textobject_around</code>	Select around object	normal: <code>ma</code> , select: <code>ma</code>

Name	Description	Default keybinds
<code>select_textobject_inner</code>	Select inside object	normal: <code>mi</code> , select: <code>mi</code>
<code>goto_next_function</code>	Goto next function	normal: <code>]f</code> , select: <code>]f</code>
<code>goto_prev_function</code>	Goto previous function	normal: <code>[f</code> , select: <code>[f</code>
<code>goto_next_class</code>	Goto next type definition	normal: <code>]t</code> , select: <code>]t</code>
<code>goto_prev_class</code>	Goto previous type definition	normal: <code>[t</code> , select: <code>[t</code>
<code>goto_next_parameter</code>	Goto next parameter	normal: <code>]a</code> , select: <code>]a</code>
<code>goto_prev_parameter</code>	Goto previous parameter	normal: <code>[a</code> , select: <code>[a</code>
<code>goto_next_comment</code>	Goto next comment	normal: <code>]c</code> , select: <code>]c</code>
<code>goto_prev_comment</code>	Goto previous comment	normal: <code>[c</code> , select: <code>[c</code>
<code>goto_next_test</code>	Goto next test	normal: <code>]T</code> , select: <code>]T</code>
<code>goto_prev_test</code>	Goto previous test	normal: <code>[T</code> , select: <code>[T</code>
<code>goto_next_entry</code>	Goto next pairing	normal: <code>]e</code> , select: <code>]e</code>
<code>goto_prev_entry</code>	Goto previous pairing	normal: <code>[e</code> , select: <code>[e</code>
<code>goto_next_paragraph</code>	Goto next paragraph	normal: <code>]p</code> , select: <code>]p</code>
<code>goto_prev_paragraph</code>	Goto previous paragraph	normal: <code>[p</code> , select: <code>[p</code>

Name	Description	Default keybinds
<code>dap_launch</code>	Launch debug target	normal: <code><space>Gl</code> , select: <code><space>Gl</code>
<code>dap_restart</code>	Restart debugging session	normal: <code><space>Gr</code> , select: <code><space>Gr</code>
<code>dap_toggle_breakpoint</code>	Toggle breakpoint	normal: <code><space>Gb</code> , select: <code><space>Gb</code>
<code>dap_continue</code>	Continue program execution	normal: <code><space>Gc</code> , select: <code><space>Gc</code>
<code>dap_pause</code>	Pause program execution	normal: <code><space>Gh</code> , select: <code><space>Gh</code>
<code>dap_step_in</code>	Step in	normal: <code><space>Gi</code> , select: <code><space>Gi</code>
<code>dap_step_out</code>	Step out	normal: <code><space>Go</code> , select: <code><space>Go</code>
<code>dap_next</code>	Step to next	normal: <code><space>Gn</code> , select: <code><space>Gn</code>
<code>dap_variables</code>	List variables	normal: <code><space>Gv</code> , select: <code><space>Gv</code>

Name	Description	Default keybinds
<code>dap_terminate</code>	End debug session	normal: <code><space>Gt</code> , select: <code><space>Gt</code>
<code>dap_edit_condition</code>	Edit breakpoint condition on current line	normal: <code><space>G<C-c></code> , select: <code><space>G<C-c></code>
<code>dap_edit_log</code>	Edit breakpoint log message on current line	normal: <code><space>G<C-l></code> , select: <code><space>G<C-l></code>
<code>dap_switch_thread</code>	Switch current thread	normal: <code><space>Gst</code> , select: <code><space>Gst</code>
<code>dap_switch_stack_frame</code>	Switch stack frame	normal: <code><space>Gsf</code> , select: <code><space>Gsf</code>
<code>dap_enable_exceptions</code>	Enable exception breakpoints	normal: <code><space>Ge</code> , select: <code><space>Ge</code>
<code>dap_disable_exceptions</code>	Disable exception breakpoints	normal: <code><space>GE</code> , select: <code><space>GE</code>
<code>shell_pipe</code>	Pipe selections through shell command	normal: <code> </code> , select: <code> </code>

Name	Description	Default keybinds
<code>shell_pipe_to</code>	Pipe selections into shell command ignoring output	normal: <code><A- ></code> , select: <code><A- ></code>
<code>shell_insert_output</code>	Insert shell command output before selections	normal: <code>!</code> , select: <code>!</code>
<code>shell_append_output</code>	Append shell command output after selections	normal: <code><A-!></code> , select: <code><A-!></code>
<code>shell_keep_pipe</code>	Filter selections with shell predicate	normal: <code>\$</code> , select: <code>\$</code>
<code>suspend</code>	Suspend and return to shell	normal: <code><C-z></code> , select: <code><C-z></code>
<code>rename_symbol</code>	Rename symbol	normal: <code><space>r</code> , select: <code><space>r</code>
<code>increment</code>	Increment item under cursor	normal: <code><C-a></code> , select: <code><C-a></code>
<code>decrement</code>	Decrement item under cursor	normal: <code><C-x></code> , select: <code><C-x></code>
<code>record_macro</code>	Record macro	normal: <code>Q</code> , select: <code>Q</code>
<code>replay_macro</code>	Replay macro	normal: <code>q</code> , select: <code>q</code>
<code>command_palette</code>	Open command palette	normal: <code><space>?</code> , select:

Name	Description	Default keybinds
		<code><space>?</code>
<code>goto_word</code>	Jump to a two-character label	normal: <code>gw</code>
<code>extend_to_word</code>	Extend to a two-character label	select: <code>gw</code>
<code>goto_next_tabstop</code>	Goto next snippet placeholder	
<code>goto_prev_tabstop</code>	Goto next snippet placeholder	
<code>rotate_selections_first</code>	Make the first selection your primary one	
<code>rotate_selections_last</code>	Make the last selection your primary one	

Language Support

The following languages and Language Servers are supported. To use Language Server features, you must first **configure** the appropriate Language Server.

You can check the language support in your installed helix version with `hx --health`.

Also see the [Language Configuration](#) docs and the [Adding Languages](#) guide for more language configuration information.

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
ada	✓	✓		<code>ada_language_server</code>
adl	✓	✓	✓	
agda	✓			
alloy	✓			
amber	✓			<code>amber-lsp</code>
astro	✓			<code>astro-ls</code>
awk	✓	✓		<code>awk-language-server</code>
bash	✓	✓	✓	<code>bash-language-server</code>
bass	✓			<code>bass</code>
beancount	✓			<code>beancount-language-server</code>
bibtex	✓			<code>texlab</code>
bicep	✓			<code>bicep-langserver</code>
bitbake	✓			<code>bitbake-language-server</code>
blade	✓			
blueprint	✓			<code>blueprint-compiler</code>
c	✓	✓	✓	<code>clangd</code>

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
c-sharp	✓	✓		Omnisharp
cabal				haskell-language- server-wrapper
caddyfile	✓	✓	✓	
cairo	✓	✓	✓	cairo-language- server
capnp	✓		✓	
cel	✓			
circom	✓			circom-lsp
clarity	✓			clarinet
clojure	✓			clojure-lsp
cmake	✓	✓	✓	neocmakelsp , cmake- language-server
codeql	✓	✓		codeql
comment	✓			
common-lisp	✓		✓	cl-lsp
cpon	✓		✓	
cpp	✓	✓	✓	clangd
crystal	✓	✓	✓	crystalline , ameba- ls
css	✓		✓	vscode-css-language- server
csv	✓			
cue	✓			cuelsp
cylc	✓	✓	✓	
d	✓	✓	✓	serve-d
dart	✓	✓	✓	dart

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
dbml	✓			
debian	✓			
devicetree	✓			dts-lsp
dhall	✓	✓		dhall-lsp-server
diff	✓			
djot	✓			
docker-compose	✓	✓	✓	docker-compose- langserver , yaml- language-server
dockerfile	✓	✓		docker-langserver
dot	✓			dot-language-server
dtd	✓			
dune	✓			
dunstrc	✓			
earthfile	✓	✓	✓	earthlyls
edoc	✓			
eex	✓			
ejs	✓			
elisp	✓			
elixir	✓	✓	✓	elixir-ls
elm	✓	✓		elm-language-server
elvish	✓			elvish
env	✓	✓		
erb	✓			
erlang	✓	✓		erlang_ls , elp
esdl	✓			

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
fennel	✓			fennel-ls
fga	✓	✓	✓	
fidl	✓			
fish	✓	✓	✓	fish-lsp
forth	✓			forth-lsp
fortran	✓		✓	fortls
fsharp	✓			fsautocomplete
gas	✓	✓		asm-lsp
gdscript	✓	✓	✓	
gemi	✓			
gherkin	✓			
ghostty	✓			
git-attributes	✓			
git-commit	✓	✓		
git-config	✓	✓		
git-ignore	✓			
git-notes	✓			
git-rebase	✓			
gjs	✓	✓	✓	typescript-language-server , vscode-eslint-language-server , ember-language-server
gleam	✓	✓		gleam
glimmer	✓			ember-language-server
glsl	✓	✓	✓	glsl_analyzer

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
gn	✓			
go	✓	✓	✓	<code>gopls</code> , <code>golangci-lint-langserver</code>
godot-resource	✓	✓		
gomod	✓			<code>gopls</code>
gotmpl	✓			<code>gopls</code>
gowork	✓			<code>gopls</code>
gpr	✓			<code>ada_language_server</code>
graphql	✓	✓		<code>graphql-lsp</code>
gren	✓	✓		
groovy	✓			
gts	✓	✓	✓	<code>typescript-language-server</code> , <code>vscode-eslint-language-server</code> , <code>ember-language-server</code>
hare	✓			
haskell	✓	✓		<code>haskell-language-server-wrapper</code>
haskell-persistent	✓			
hcl	✓	✓	✓	<code>terraform-ls</code>
heex	✓	✓		<code>elixir-ls</code>
helm	✓			<code>helm_ls</code>
hocon	✓	✓	✓	
hoon	✓			
hosts	✓			

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
html	✓			<code>vscode-html-language-server</code> , <code>superhtml</code>
htmldjango	✓			<code>djlsp</code> , <code>vscode-html-language-server</code> , <code>superhtml</code>
hurl	✓	✓	✓	
hyprlang	✓		✓	<code>hyprls</code>
idris				<code>idris2-lsp</code>
iex	✓			
ini	✓			
ink	✓			
inko	✓	✓	✓	
janet	✓			
java	✓	✓	✓	<code>jdtls</code>
javascript	✓	✓	✓	<code>typescript-language-server</code>
jinja	✓			
jjdescription	✓			
jq	✓	✓		<code>jq-lsp</code>
jsdoc	✓			
json	✓	✓	✓	<code>vscode-json-language-server</code>
json-ld	✓	✓	✓	<code>vscode-json-language-server</code>
json5	✓			
jsonc	✓		✓	<code>vscode-json-language-server</code>

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
jsonnet	✓			jsonnet-language-server
jsx	✓	✓	✓	typescript-language-server
julia	✓	✓	✓	julia
just	✓	✓	✓	just-lsp
kdl	✓	✓	✓	
koka	✓		✓	koka
kotlin	✓	✓	✓	kotlin-language-server
koto	✓	✓	✓	koto-ls
latex	✓	✓		texlab
ld	✓		✓	
ldif	✓			
lean	✓			lean
ledger	✓			
llvm	✓	✓	✓	
llvm-mir	✓	✓	✓	
llvm-mir-yaml	✓		✓	
log	✓			
lpf	✓			
lua	✓	✓	✓	lua-language-server
luau	✓	✓	✓	luau-lsp
mail	✓	✓		
make	✓		✓	
markdoc	✓			markdoc-ls

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
markdown	✓			marksman, markdown-oxide
markdown-rustdoc	✓			
markdown.inline	✓			
matlab	✓	✓	✓	
mermaid	✓			
meson	✓		✓	mesonlsp
mint				mint
mojo	✓	✓	✓	pixi
move	✓			
msbuild	✓		✓	
nasm	✓	✓		asm-lsp
nestedtext	✓	✓	✓	
nginx	✓			
nickel	✓		✓	nls
nim	✓	✓	✓	nimlangserver
nix	✓	✓	✓	nil, nixd
nu	✓			nu
nunjucks	✓			
ocaml	✓		✓	ocamllsp
ocaml-interface	✓			ocamllsp
odin	✓	✓	✓	ols
ohm	✓	✓	✓	
openc1	✓	✓	✓	clangd
openscad	✓			openscad-lsp

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
org	✓			
pascal	✓	✓		<code>pasls</code>
passwd	✓			
pem	✓			
perl	✓	✓	✓	<code>perlnavigator</code>
pest	✓	✓	✓	<code>pest-language-server</code>
php	✓	✓	✓	<code>intelephense</code>
php-only	✓			
pkgbuild	✓	✓	✓	<code>termux-language-server</code> , <code>bash-language-server</code>
pkl	✓		✓	<code>pkl-lsp</code>
po	✓	✓		
pod	✓			
ponylang	✓	✓	✓	
powershell	✓			
prisma	✓	✓		<code>prisma-language-server</code>
prolog	✓		✓	<code>swipl</code>
properties	✓	✓		
protobuf	✓	✓	✓	<code>buf</code> , <code>pb</code> , <code>protols</code>
prql	✓			
pug	✓			
purescript	✓	✓		<code>purescript-language-server</code>

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
python	✓	✓	✓	ty, ruff, jedi-language-server, pylsp
qml	✓	✓	✓	qmls
quarto	✓		✓	
quint	✓			quint-language-server
r	✓			R
racket	✓		✓	racket
regex	✓			
rego	✓			regols
rescript	✓	✓		rescript-language-server
rmarkdown	✓		✓	R
robot	✓			robotframework_ls
ron	✓		✓	
rst	✓			
ruby	✓	✓	✓	ruby-lsp, solargraph
rust	✓	✓	✓	rust-analyzer
rust-format-args	✓			
rust-format-args-macro	✓	✓	✓	
sage	✓	✓		
scala	✓	✓	✓	metals
scheme	✓		✓	

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
scss	✓			vscode-css-language-server
slang	✓	✓	✓	slangd
slint	✓	✓	✓	slint-lsp
smali	✓		✓	
smithy	✓			cs
sml	✓			
snakemake	✓		✓	pylsp
solidity	✓	✓		solc
sourcepawn	✓	✓		sourcepawn-studio
spade	✓		✓	spade-language-server
spicedb	✓			
sql	✓	✓		
sshclientconfig	✓			
starlark	✓	✓	✓	starpls
strace	✓			
supercollider	✓			
svelte	✓		✓	svelte-server
sway	✓	✓	✓	forc
swift	✓	✓		sourcekit-lsp
systemd	✓			systemd-lsp
t32	✓			
tablegen	✓	✓	✓	
tact	✓	✓	✓	
task	✓			

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
tcl	✓		✓	
teal	✓			teal-language-server
templ	✓			templ
tera	✓			
textproto	✓	✓	✓	
tfvars	✓		✓	terraform-ls
thrift	✓			
tlaplus	✓			
todotxt	✓			
toml	✓	✓		taplo , tombi
tsq	✓			ts_query_ls
tsx	✓	✓	✓	typescript-language-server
twig	✓			
typescript	✓	✓	✓	typescript-language-server
typespec	✓	✓	✓	tsp-server
typst	✓			tinymist
ungrammar	✓			
unison	✓	✓	✓	
uxntal	✓			
v	✓	✓	✓	v-analyzer
vala	✓	✓		vala-language-server
vento	✓			
verilog	✓	✓		svlangserver
vhdl	✓			vhdl_ls

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Default language servers
vhs	✓			
vue	✓			vue-language-server
wast	✓			
wat	✓			wat_server
webc	✓			
werk	✓			
wesl	✓	✓		
wgsl	✓			wgsl-analyzer
wit	✓		✓	
wren	✓	✓	✓	
xit	✓			
xml	✓		✓	
xtc	✓			
yaml	✓	✓	✓	yaml-language-server, ansible-language-server
yara	✓			yls
yuck	✓			
zig	✓	✓	✓	zls

Ecosystem

This section has information related to the wider Helix ecosystem.

Migrating from Vim

Helix's editing model is strongly inspired from Vim and Kakoune, and a notable difference from Vim (and the most striking similarity to Kakoune) is that Helix follows the `selection → action` model. This means that whatever you are going to act on (a word, a paragraph, a line, etc.) is selected first and the action itself (delete, change, yank, etc.) comes second. A cursor is simply a single width selection.

See also Kakoune's [Migrating from Vim](#) and Helix's [Migrating from Vim](#).

TODO: Mention textobjects, surround, registers

Helix mode in other software

Helix' keymap and interaction model ([Using Helix](#)) is easier to adopt if it can be used consistently in many editing contexts. Yet, certain use cases cannot easily be addressed directly in Helix. Similar to vim, this leads to the creation of "Helix mode" in various other software products, allowing Helix-style editing for a greater variety of use cases.

"Helix mode" is frequently still in early stages or missing entirely. For such cases, we also link to relevant bugs or discussions.

Other editors

Editor	Plugin or feature providing Helix editing	Comments
Vim	<code>helix.vim</code> config	
IntelliJ IDEA / Android Studio	IdeaVim plugin + <code>helix.idea.vim</code> config	Minimum recommended version is IdeaVim 2.19.0.
Visual Studio	VsVim plugin + <code>helix.vs.vim</code> config	
Visual Studio Code	Dance extension, or its Helix fork	The Helix fork has diverged. You can also use the original Dance and tweak its keybindings directly (try this config).
Visual Studio Code	Helix for VS Code extension	
Zed	native via keybindings (Bug)	
CodeMirror	<code>codemirror-helix</code>	

Shells

Shell	Plugin or feature providing Helix editing
Fish	Feature Request
Fish	<code>fish-helix</code>
Zsh	<code>helix-zsh</code> or <code>zsh-helix-mode</code>
Nushell	Feature Request

Other software

Software	Plugin or feature providing Helix editing.	Comments
Obsidian	Obsidian-Helix	Uses <code>codemirror-helix</code> listed above.

Configuration

To override global configuration parameters, create a `config.toml` file located in your config directory:

- Linux and Mac: `~/.config/helix/config.toml`
- Windows: `%AppData%\helix\config.toml`

💡 You can easily open the config file by typing `:config-open` within Helix normal mode.

Example config:

```
theme = "onedark"

[editor]
line-number = "relative"
mouse = false

[editor.cursor-shape]
insert = "bar"
normal = "block"
select = "underline"

[editor.file-picker]
hidden = false
```

You can use a custom configuration file by specifying it with the `-c` or `--config` command line argument, for example `hx -c path/to/custom-config.toml`. You can reload the config file by issuing the `:config-reload` command. Alternatively, on Unix operating systems, you can reload it by sending the USR1 signal to the Helix process, such as by using the command `pkill -USR1 hx`.

Finally, you can have a `config.toml` local to a project by putting it under a `.helix` directory in your repository. Its settings will be merged with the configuration directory `config.toml` and the built-in configuration.

Editor

- `[editor]` Section
- `[editor.clipboard-provider]` Section
- `[editor.statusline]` Section
- `[editor.lsp]` Section
- `[editor.cursor-shape]` Section
- `[editor.file-picker]` Section
- `[editor.auto-pairs]` Section
- `[editor.auto-save]` Section
- `[editor.search]` Section
- `[editor.whitespace]` Section
- `[editor.indent-guides]` Section
- `[editor.gutters]` Section
 - `[editor.gutters.line-numbers]` Section
 - `[editor.gutters.diagnostics]` Section
 - `[editor.gutters.diff]` Section
 - `[editor.gutters.spacer]` Section
- `[editor.soft-wrap]` Section
- `[editor.smart-tab]` Section
- `[editor.inline-diagnostics]` Section

`[editor]` Section

Key	Description	Default
<code>scrolloff</code>	Number of lines of padding around the edge of the screen when scrolling	<code>5</code>
<code>mouse</code>	Enable mouse mode	<code>true</code>
<code>default-yank-register</code>	Default register used for yank/paste	<code>"</code>
<code>middle-click-paste</code>	Middle click paste support	<code>true</code>

Key	Description	Default
<code>scroll-lines</code>	Number of lines to scroll per scroll wheel step	<code>3</code>
<code>shell</code>	Shell to use when running external commands	Unix: <code>["sh", "-c"]</code> Windows: <code>["cmd", "/C"]</code>
<code>line-number</code>	Line number display: <code>absolute</code> simply shows each line's number, while <code>relative</code> shows the distance from the current line. When unfocused or in insert mode, <code>relative</code> will still show absolute line numbers	<code>absolute</code>
<code>cursorline</code>	Highlight all lines with a cursor	<code>false</code>
<code>cursorcolumn</code>	Highlight all columns with a cursor	<code>false</code>
<code>continue-comments</code>	if helix should automatically add a line comment token if you create a new line inside a comment.	<code>true</code>
<code>gutters</code>	Gutters to display: Available are <code>diagnostics</code> and <code>diff</code> and <code>line-numbers</code> and <code>spacer</code> , note that <code>diagnostics</code> also includes other features like breakpoints, 1-width padding will be inserted if gutters is non-empty	<code>["diagnostics", "spacer", "line-numbers", "spacer", "diff"]</code>
<code>auto-completion</code>	Enable automatic pop up of auto-completion	<code>true</code>
<code>path-completion</code>	Enable filepath completion. Show files and directories if an existing path at the cursor was recognized, either absolute or relative to the current opened document or current working directory (if the buffer is	<code>true</code>

Key	Description	Default
	not yet saved). Defaults to true.	
<code>auto-format</code>	Enable automatic formatting on save	<code>true</code>
<code>idle-timeout</code>	Time in milliseconds since last keypress before idle timers trigger.	<code>250</code>
<code>completion-timeout</code>	Time in milliseconds after typing a word character before completions are shown, set to 5 for instant.	<code>250</code>
<code>preview-completion-insert</code>	Whether to apply completion item instantly when selected	<code>true</code>
<code>completion-trigger-len</code>	The min-length of word under cursor to trigger autocompletion	<code>2</code>
<code>completion-replace</code>	Set to <code>true</code> to make completions always replace the entire word and not just the part before the cursor	<code>false</code>
<code>auto-info</code>	Whether to display info boxes	<code>true</code>
<code>true-color</code>	Set to <code>true</code> to override automatic detection of terminal truecolor support in the event of a false negative	<code>false</code>
<code>undercurl</code>	Set to <code>true</code> to override automatic detection of terminal undercurl support in the event of a false negative	<code>false</code>

Key	Description	Default
<code>rulers</code>	List of column positions at which to display the rulers. Can be overridden by language specific <code>rulers</code> in <code>languages.toml</code> file	<code>[]</code>
<code>bufferline</code>	Renders a line at the top of the editor displaying open buffers. Can be <code>always</code> , <code>never</code> or <code>multiple</code> (only shown if more than one buffer is in use)	<code>never</code>
<code>color-modes</code>	Whether to color the mode indicator with different colors depending on the mode itself	<code>false</code>
<code>text-width</code>	Maximum line length. Used for the <code>:reflow</code> command and soft-wrapping if <code>soft-wrap.wrap-at-text-width</code> is set	<code>80</code>
<code>workspace-lsp-roots</code>	Directories relative to the workspace root that are treated as LSP roots. Should only be set in <code>.helix/config.toml</code>	<code>[]</code>
<code>default-line-ending</code>	The line ending to use for new documents. Can be <code>native</code> , <code>lf</code> , <code>crlf</code> , <code>ff</code> , <code>cr</code> or <code>nel</code> . <code>native</code> uses the platform's native line ending (<code>crlf</code> on Windows, otherwise <code>lf</code>).	<code>native</code>
<code>insert-final-newline</code>	Whether to automatically insert a trailing line-ending on write if missing	<code>true</code>
<code>atomic-save</code>	Whether to use atomic operations to write documents to disk. This prevents data loss if the editor is interrupted while writing the file, but may	<code>true</code>

Key	Description	Default
	confuse some file watching/ hot reloading programs.	
<code>trim-final-newlines</code>	Whether to automatically remove line-endings after the final one on write	<code>false</code>
<code>trim-trailing-whitespace</code>	Whether to automatically remove whitespace preceding line endings on write	<code>false</code>
<code>popup-border</code>	Draw border around <code>popup</code> , <code>menu</code> , <code>all</code> , or <code>none</code>	<code>none</code>
<code>indent-heuristic</code>	How the indentation for a newly inserted line is computed: <code>simple</code> just copies the indentation level from the previous line, <code>tree-sitter</code> computes the indentation based on the syntax tree and <code>hybrid</code> combines both approaches. If the chosen heuristic is not available, a different one will be used as a fallback (the fallback order being <code>hybrid</code> -> <code>tree-sitter</code> -> <code>simple</code>).	<code>hybrid</code>
<code>jump-label-alphabet</code>	The characters that are used to generate two character jump labels. Characters at the start of the alphabet are used first.	<code>"abcdefghijklmnopqrstuvwxyz"</code>
<code>end-of-line-diagnostics</code>	Minimum severity of diagnostics to render at the end of the line. Set to <code>disable</code> to disable entirely. Refer to the setting about <code>inline-diagnostics</code> for more details	<code>"disable"</code>
<code>clipboard-provider</code>	Which API to use for clipboard interaction. One of <code>pasteboard</code> (MacOS),	Platform and environment specific.

Key	Description	Default
	wayland, x-clip, x-sel, win-32-yank, termux, tmux, windows, termcode, none, or a custom command set.	
editor-config	Whether to read settings from EditorConfig files	true

[editor.clipboard-provider] Section

Helix can be configured either to use a builtin clipboard configuration or to use a provided command.

For instance, setting it to use OSC 52 termcodes, the configuration would be:

```
[editor]
clipboard-provider = "termcode"
```

Alternatively, Helix can be configured to use arbitrary commands for clipboard integration:

```
[editor.clipboard-provider.custom]
yank = { command = "cat", args = ["test.txt"] }
paste = { command = "tee", args = ["test.txt"] }
primary-yank = { command = "cat", args = ["test-primary.txt"] } # optional
primary-paste = { command = "tee", args = ["test-primary.txt"] } # optional
```

For custom commands the contents of the yank/paste is communicated over stdin/stdout.

[editor.statusline] Section

Allows configuring the statusline at the bottom of the editor.

The configuration distinguishes between three areas of the status line:

```
[ ... .. LEFT ... .. | ... .. CENTER ... .. | ... .. RIGHT ... ..
]
```

Statusline elements can be defined as follows:

```
[editor.statusline]
left = ["mode", "spinner"]
center = ["file-name"]
right = ["diagnostics", "selections", "position", "file-encoding", "file-line-ending", "file-type"]
separator = "|"
mode.normal = "NORMAL"
mode.insert = "INSERT"
mode.select = "SELECT"
diagnostics = ["warning", "error"]
workspace-diagnostics = ["warning", "error"]
```

The `[editor.statusline]` key takes the following sub-keys:

Key	Description	Default
<code>left</code>	A list of elements aligned to the left of the statusline	<code>["mode", "spinner", "file-name", "read-only-indicator", "file-modification-indicator"]</code>
<code>center</code>	A list of elements aligned to the middle of the statusline	<code>[]</code>
<code>right</code>	A list of elements aligned to the right of the statusline	<code>["diagnostics", "selections", "register", "position", "file-encoding"]</code>
<code>separator</code>	The character used to separate elements in the statusline	<code>" "</code>
<code>mode.normal</code>	The text shown in the <code>mode</code> element for normal mode	<code>"NOR"</code>
<code>mode.insert</code>	The text shown in the <code>mode</code> element for insert mode	<code>"INS"</code>
<code>mode.select</code>	The text shown in the <code>mode</code> element for select mode	<code>"SEL"</code>

Key	Description	Default
<code>diagnostics</code>	A list of severities which are displayed for the current buffer	<code>["warning", "error"]</code>
<code>workspace-diagnostics</code>	A list of severities which are displayed for the workspace	<code>["warning", "error"]</code>

The following statusline elements can be configured:

Key	Description
<code>mode</code>	The current editor mode (<code>mode.normal</code> / <code>mode.insert</code> / <code>mode.select</code>)
<code>spinner</code>	A progress spinner indicating LSP activity
<code>file-name</code>	The path/name of the opened file
<code>file-absolute-path</code>	The absolute path/name of the opened file
<code>file-base-name</code>	The basename of the opened file
<code>file-modification-indicator</code>	The indicator to show whether the file is modified (a <code>[+]</code> appears when there are unsaved changes)
<code>file-encoding</code>	The encoding of the opened file if it differs from UTF-8
<code>file-line-ending</code>	The file line endings (CRLF or LF)
<code>file-indent-style</code>	The file indentation style
<code>read-only-indicator</code>	An indicator that shows <code>[readonly]</code> when a file cannot be written
<code>total-line-numbers</code>	The total line numbers of the opened file
<code>file-type</code>	The type of the opened file
<code>diagnostics</code>	The number of warnings and/or errors
<code>workspace-diagnostics</code>	The number of warnings and/or errors on workspace

Key	Description
<code>selections</code>	The primary selection index out of the number of active selections
<code>primary-selection-length</code>	The number of characters currently in primary selection
<code>position</code>	The cursor position
<code>position-percentage</code>	The cursor position as a percentage of the total number of lines
<code>separator</code>	The string defined in <code>editor.statusline.separator</code> (defaults to <code>" "</code>)
<code>spacer</code>	Inserts a space between elements (multiple/contiguous spacers may be specified)
<code>version-control</code>	The current branch name or detached commit hash of the opened workspace
<code>register</code>	The current selected register

[`editor.lsp`] Section

Key	Description	Default
<code>enable</code>	Enables LSP integration. Setting to false will completely disable language servers regardless of language settings.	<code>true</code>
<code>display-messages</code>	Display LSP <code>window/showMessage</code> messages below statusline ¹	<code>true</code>
<code>display-progress-messages</code>	Display LSP progress messages below statusline ¹	<code>false</code>
<code>auto-signature-help</code>	Enable automatic popup of signature help (parameter hints)	<code>true</code>
<code>display-inlay-hints</code>	Display inlay hints ²	<code>false</code>
<code>inlay-hints-length-limit</code>	Maximum displayed length (non-zero number) of inlay hints	Unset by default

Key	Description	Default
<code>display-color-swatches</code>	Show color swatches next to colors	<code>true</code>
<code>display-signature-help-docs</code>	Display docs under signature help popup	<code>true</code>
<code>snippets</code>	Enables snippet completions. Requires a server restart (<code>:lsp-restart</code>) to take effect after <code>:config-reload</code> / <code>:set</code> .	<code>true</code>
<code>goto-reference-include-declaration</code>	Include declaration in the goto references popup.	<code>true</code>

¹ By default, a progress spinner is shown in the statusline beside the file path.

² You may also have to activate them in the language server config for them to appear, not just in Helix. Inlay hints in Helix are still being improved on and may be a little bit laggy/janky under some circumstances. Please report any bugs you see so we can fix them!

[editor.cursor-shape] Section

Defines the shape of cursor in each mode. Valid values for these options are `block`, `bar`, `underline`, or `hidden`.

💡 Due to limitations of the terminal environment, only the primary cursor can change shape.

Key	Description	Default
<code>normal</code>	Cursor shape in normal mode	<code>block</code>
<code>insert</code>	Cursor shape in insert mode	<code>block</code>
<code>select</code>	Cursor shape in select mode	<code>block</code>

[editor.file-picker] Section

Set options for file picker and global search. Ignoring a file means it is not visible in the Helix file picker and global search.

All git related options are only enabled in a git repository.

Key	Description	Default
<code>hidden</code>	Enables ignoring hidden files	<code>true</code>
<code>follow-symlinks</code>	Follow symlinks instead of ignoring them	<code>true</code>
<code>deduplicate-links</code>	Ignore symlinks that point at files already shown in the picker	<code>true</code>
<code>parents</code>	Enables reading ignore files from parent directories	<code>true</code>
<code>ignore</code>	Enables reading <code>.ignore</code> files	<code>true</code>
<code>git-ignore</code>	Enables reading <code>.gitignore</code> files	<code>true</code>
<code>git-global</code>	Enables reading global <code>.gitignore</code> , whose path is specified in git's config: <code>core.excludesfile</code> option	<code>true</code>
<code>git-exclude</code>	Enables reading <code>.git/info/exclude</code> files	<code>true</code>
<code>max-depth</code>	Set with an integer value for maximum depth to recurse	Unset by default

Ignore files can be placed locally as `.ignore` or put in your home directory as `~/.ignore`. They support the usual ignore and negative ignore (unignore) rules used in `.gitignore` files.

Additionally, you can use Helix-specific ignore files by creating a local `.helix/ignore` file in the current workspace or a global `ignore` file located in your Helix config directory:

- Linux and Mac: `~/.config/helix/ignore`
- Windows: `%AppData%\helix\ignore`

Example:

```
# unignore in file picker and global search
!.github/
!.gitignore
!.gitattributes
```

[editor.auto-pairs] Section

Enables automatic insertion of pairs to parentheses, brackets, etc. Can be a simple boolean value, or a specific mapping of pairs of single characters.

To disable auto-pairs altogether, set `auto-pairs` to `false`:

```
[editor]
auto-pairs = false # defaults to `true`
```

The default pairs are `(){}[]'""`<`>`, but these can be customized by setting `auto-pairs` to a TOML table:

```
[editor.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
'`' = '`'
'<' = '>'
```

Additionally, this setting can be used in a language config. Unless the editor setting is `false`, this will override the editor config in documents with this language.

Example `languages.toml` that adds `<>` and removes `' '`

```
[[language]]
name = "rust"

[language.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
'`' = '`'
'<' = '>'
```

[editor.auto-save] Section

Control auto save behavior.

Key	Description	Default
<code>focus-lost</code>	Enable automatic saving on the focus moving away from Helix. Requires <code>focus event</code> support from your terminal	<code>false</code>
<code>after-delay.enable</code>	Enable automatic saving after <code>auto-save.after-delay.timeout</code> milliseconds have passed since last edit.	<code>false</code>
<code>after-delay.timeout</code>	Time in milliseconds since last edit before auto save timer triggers.	<code>3000</code>

[editor.search] Section

Search specific options.

Key	Description	Default
<code>smart-case</code>	Enable smart case regex searching (case-insensitive unless pattern contains upper case characters)	<code>true</code>
<code>wrap-around</code>	Whether the search should wrap after depleting the matches	<code>true</code>

[editor.whitespace] Section

Options for rendering whitespace with visible characters. Use `:set whitespace.render all` to temporarily enable visible whitespace.

Key	Description	Default
<code>render</code>	Whether to render whitespace. May either be <code>all</code> or <code>none</code> , or a table with sub-keys <code>space</code> , <code>nbsp</code> , <code>nnbsp</code> , <code>tab</code> , and <code>newline</code>	<code>none</code>
<code>characters</code>	Literal characters to use when rendering whitespace. Sub-keys may be any of <code>tab</code> , <code>space</code> ,	See example

Example

[editor.indent-guides] Section

Options for rendering vertical indent guides.

Example:

[editor.gutters] Section

For simplicity, `editor.gutters` accepts an array of gutter types, which will use default settings for all gutter components.

```
[editor]
gutters = ["diff", "diagnostics", "line-numbers", "spacer"]
```

To customize the behavior of gutters, the `[editor.gutters]` section must be used. This section contains top level settings, as well as settings for specific gutter components as subsections.

Key	Description	Default
<code>layout</code>	A vector of gutters to display	<code>["diagnostics", "spacer", "line-numbers", "spacer", "diff"]</code>

Example:

```
[editor.gutters]
layout = ["diff", "diagnostics", "line-numbers", "spacer"]
```

[editor.gutters.line-numbers] Section

Options for the line number gutter

Key	Description	Default
<code>min-width</code>	The minimum number of characters to use	<code>3</code>

Example:

```
[editor.gutters.line-numbers]
min-width = 1
```

[editor.gutters.diagnostics] Section

Currently unused

[editor.gutters.diff] Section

The `diff` gutter option displays colored bars indicating whether a `git` diff represents that a line was added, removed or changed. These colors are controlled by the theme attributes `diff.plus`, `diff.minus` and `diff.delta`.

Other diff providers will eventually be supported by a future plugin system.

There are currently no options for this section.

[editor.gutters.spacer] Section

Currently unused

[editor.soft-wrap] Section

Options for soft wrapping lines that exceed the view width:

Key	Description	Default
<code>enable</code>	Whether soft wrapping is enabled.	<code>false</code>
<code>max-wrap</code>	Maximum free space left at the end of the line.	<code>20</code>
<code>max-indent-retain</code>	Maximum indentation to carry over when soft wrapping a line.	<code>40</code>
<code>wrap-indicator</code>	Text inserted before soft wrapped lines, highlighted with <code>ui.virtual.wrap</code>	<code>↩</code>
<code>wrap-at-text-width</code>	Soft wrap at <code>text-width</code> instead of using the full viewport size.	<code>false</code>

Example:

```
[editor.soft-wrap]
enable = true
max-wrap = 25 # increase value to reduce forced mid-word wrapping
max-indent-retain = 0
wrap-indicator = "" # set wrap-indicator to "" to hide it
```

[editor.smart-tab] Section

Options for navigating and editing using tab key.

Key	Description	Default
<code>enable</code>	If set to true, then when the cursor is in a position with non-whitespace to its left, instead of inserting a tab, it will run <code>move_parent_node_end</code> . If there is only whitespace to the left, then it inserts a tab as normal. With the default bindings, to explicitly insert a tab character, press Shift-tab.	<code>true</code>
<code>supersede-menu</code>	Normally, when a menu is on screen, such as when auto complete is triggered, the tab key is bound to cycling through the items. This means when menus are on screen, one cannot use the tab key to trigger the <code>smart-tab</code> command. If this option is set to true, the <code>smart-tab</code> command always takes precedence, which means one cannot use the tab key to cycle through menu items. One of the other bindings must be used instead, such as arrow keys or <code>C-n</code> / <code>C-p</code> .	<code>false</code>

Due to lack of support for S-tab in some terminals, the default keybindings don't fully embrace smart-tab editing experience. If you enjoy smart-tab navigation and a terminal that supports the Enhanced Keyboard protocol, consider setting extra keybindings:

```
[keys.normal]
tab = "move_parent_node_end"
S-tab = "move_parent_node_start"

[keys.insert]
S-tab = "move_parent_node_start"

[keys.select]
tab = "extend_parent_node_end"
S-tab = "extend_parent_node_start"
```

[editor.inline-diagnostics] Section

Options for rendering diagnostics inside the text like shown below

```
fn main() {
  let foo = bar;
    └─ no such value in this scope
}
```

Key	Description	Default
<code>cursor-line</code>	The minimum severity that a diagnostic must have to be shown inline on the line that contains the primary cursor. Set to <code>disable</code> to not show any diagnostics inline. This option does not have any effect when in insert-mode and will only take effect 350ms after moving the cursor to a different line.	<code>"disable"</code>
<code>other-lines</code>	The minimum severity that a diagnostic must have to be shown inline on a line that does not contain the cursor-line. Set to <code>disable</code> to not show any diagnostics inline.	<code>"disable"</code>
<code>prefix-len</code>	How many horizontal bars <code>—</code> are rendered before the diagnostic text.	<code>1</code>
<code>max-wrap</code>	Equivalent of the <code>editor.soft-wrap.max-wrap</code> option for diagnostics.	<code>20</code>
<code>max-diagnostics</code>	Maximum number of diagnostics to render inline for a given line	<code>10</code>

The allowed values for `cursor-line` and `other-lines` are: `error`, `warning`, `info`, `hint`.

The (first) diagnostic with the highest severity that is not shown inline is rendered at the end of the line (as long as its severity is higher than the `end-of-line-diagnostics` config option):

```
fn main() {
  let baz = 1;
  let foo = bar; a local variable with a similar name exists: baz
    └─ no such value in this scope
}
```

The new diagnostic rendering is not yet enabled by default. As soon as end of line or inline diagnostics are enabled the old diagnostics rendering is automatically disabled. The recommended default setting

are:

```
[editor]
end-of-line-diagnostics = "hint"
[editor.inline-diagnostics]
cursor-line = "warning" # show warnings and errors on the cursorline inline
```

Themes

To use a theme add `theme = "<name>"` to the top of your `config.toml` file, or select it during runtime using `:theme <name>`.

Creating a theme

Create a file with the name of your theme as the file name (i.e `mytheme.toml`) and place it in your `themes` directory (i.e `~/.config/helix/themes` or `%AppData%\helix\themes` on Windows). The directory might have to be created beforehand.

💡 The names "default" and "base16_default" are reserved for built-in themes and cannot be overridden by user-defined themes.

Overview

Each line in the theme file is specified as below:

```
key = { fg = "#ffffff", bg = "#000000", underline = { color = "#ff0000", style = "curl"}, modifiers = ["bold", "italic"] }
```

Where `key` represents what you want to style, `fg` specifies the foreground color, `bg` the background color, `underline` the underline style / color, and `modifiers` is a list of style modifiers. `bg`, `underline` and `modifiers` can be omitted to defer to the defaults.

To specify only the foreground color:

```
key = "#ffffff"
```

If the key contains a dot `'.'`, it must be quoted to prevent it being parsed as a dotted key.

```
"key.key" = "#ffffff"
```

For inspiration, you can find the default `theme.toml` [here](#) and user-submitted themes [here](#).

The details of theme creation

Color palettes

It's recommended to define a palette of named colors, and refer to them in the configuration values in your theme. To do this, add a table called `palette` to your theme file:

```
"ui.background" = "white"
"ui.text" = "black"

[palette]
white = "#ffffff"
black = "#000000"
```

Keep in mind that the `[palette]` table includes all keys after its header, so it should be defined after the normal theme options.

The default palette uses the terminal's default 16 colors, and the colors names are listed below. The `[palette]` section in the config file takes precedence over it and is merged into the default palette.

Color Name

`default`

`black`

`red`

`green`

`yellow`

`blue`

`magenta`

`cyan`

`gray`

`light-red`

`light-green`

Color Name

`light-yellow`

`light-blue`

`light-magenta`

`light-cyan`

`light-gray`

`white`

Modifiers

The following values may be used as modifier, provided they are supported by your terminal emulator.

Modifier

`bold`

`dim`

`italic`

`underlined`

`slow_blink`

`rapid_blink`

`reversed`

`hidden`

`crossed_out`



The `underlined` modifier is deprecated and only available for backwards compatibility. Its behavior is equivalent to setting `underline.style="line"`.

Underline style

One of the following values may be used as a value for `underline.style`, providing it is supported by your terminal emulator.

Modifier

`line`

`curl`

`dashed`

`dotted`

`double_line`

Inheritance

Extend other themes by setting the `inherits` property to an existing theme.

```
inherits = "boo_berry"

# Override the theming for "keyword"s:
"keyword" = { fg = "gold" }

# Override colors in the palette:
[palette]
berry = "#2A2A4D"
```

Scopes

The following is a list of scopes available to use for styling:

Syntax highlighting

These keys match `tree-sitter` scopes.

When determining styling for a highlight, the longest matching theme key will be used. For example, if the highlight is `function.builtin.static`, the key `function.builtin` will be used instead of `function`.

We use a similar set of scopes as Sublime Text. See also TextMate

scopes.

- `attribute` - Class attributes, HTML tag attributes
- `type` - Types
 - `builtin` - Primitive types provided by the language (`int`, `usize`)
 - `parameter` - Generic type parameters (`T`)
 - `enum`
 - `variant`
- `constructor`
- `constant` (TODO: constant.other.placeholder for `%v`)
 - `builtin` Special constants provided by the language (`true`, `false`, `nil` etc)
 - `boolean`
 - `character`
 - `escape`
 - `numeric` (numbers)
 - `integer`
 - `float`
- `string` (TODO: string.quoted.{single, double}, string.raw/.unquoted)?
 - `regexp` - Regular expressions
 - `special`
 - `path`
 - `url`
 - `symbol` - Erlang/Elixir atoms, Ruby symbols, Clojure keywords
- `comment` - Code comments
 - `line` - Single line comments (`//`)
 - `documentation` - Line documentation comments (e.g. `///` in Rust)
 - `block` - Block comments (e.g. `/* */`)
 - `documentation` - Block documentation comments (e.g. `/** */` in Rust)
 - `unused` - Unused variables and patterns, e.g. `_` and `_foo`
- `variable` - Variables

- `builtin` - Reserved language variables (`self`, `this`, `super`, etc.)
- `parameter` - Function parameters
- `other`
 - `member` - Fields of composite data types (e.g. structs, unions)
 - `private` - Private fields that use a unique syntax (currently just ECMAScript-based languages)
- `label` - `.class`, `#id` in CSS, etc.
- `punctuation`
 - `delimiter` - Commas, colons
 - `bracket` - Parentheses, angle brackets, etc.
 - `special` - String interpolation brackets.
- `keyword`
 - `control`
 - `conditional` - `if`, `else`
 - `repeat` - `for`, `while`, `loop`
 - `import` - `import`, `export`
 - `return`
 - `exception`
 - `operator` - `or`, `in`
 - `directive` - Preprocessor directives (`#if` in C)
 - `function` - `fn`, `func`
 - `storage` - Keywords describing how things are stored
 - `type` - The type of something, `class`, `function`, `var`, `let`, etc.
 - `modifier` - Storage modifiers like `static`, `mut`, `const`, `ref`, etc.
- `operator` - `||`, `+=`, `>`
- `function`
 - `builtin`
 - `method`
 - `private` - Private methods that use a unique syntax (currently just ECMAScript-based languages)
 - `macro`

- `special` (preprocessor in C)
- `tag` - Tags (e.g. `<body>` in HTML)
 - `builtin`
- `namespace`
- `special` - `derive` in Rust, etc.
- `markup`
 - `heading`
 - `marker`
 - `1`, `2`, `3`, `4`, `5`, `6` - heading text for h1 through h6
 - `list`
 - `unnumbered`
 - `numbered`
 - `checked`
 - `unchecked`
 - `bold`
 - `italic`
 - `strikethrough`
 - `link`
 - `url` - URLs pointed to by links
 - `label` - non-URL link references
 - `text` - URL and image descriptions in links
 - `quote`
 - `raw`
 - `inline`
 - `block`
- `diff` - version control changes
 - `plus` - additions
 - `gutter` - gutter indicator
 - `minus` - deletions
 - `gutter` - gutter indicator
 - `delta` - modifications
 - `moved` - renamed or moved files/changes
 - `conflict` - merge conflicts
 - `gutter` - gutter indicator

Interface

These scopes are used for theming the editor interface:

- `markup`
 - `normal`
 - `completion` - for completion doc popup UI
 - `hover` - for hover popup UI
 - `heading`
 - `completion` - for completion doc popup UI
 - `hover` - for hover popup UI
 - `raw`
 - `inline`
 - `completion` - for completion doc popup UI
 - `hover` - for hover popup UI

Key	Notes
<code>ui.background</code>	
<code>ui.background.separator</code>	Picker separator below input line
<code>ui.cursor</code>	
<code>ui.cursor.normal</code>	
<code>ui.cursor.insert</code>	
<code>ui.cursor.select</code>	
<code>ui.cursor.match</code>	Matching bracket etc.
<code>ui.cursor.primary</code>	Cursor with primary selection
<code>ui.cursor.primary.normal</code>	
<code>ui.cursor.primary.insert</code>	
<code>ui.cursor.primary.select</code>	
<code>ui.debug.breakpoint</code>	Breakpoint indicator, found in the gutter
<code>ui.debug.active</code>	Indicator for the line at which debugging execution is paused at, found in the gutter

Key	Notes
<code>ui.gutter</code>	Gutter
<code>ui.gutter.selected</code>	Gutter for the line the cursor is on
<code>ui.linenr</code>	Line numbers
<code>ui.linenr.selected</code>	Line number for the line the cursor is on
<code>ui.statusline</code>	Statusline
<code>ui.statusline.inactive</code>	Statusline (unfocused document)
<code>ui.statusline.normal</code>	Statusline mode during normal mode (only if <code>editor.color-modes</code> is enabled)
<code>ui.statusline.insert</code>	Statusline mode during insert mode (only if <code>editor.color-modes</code> is enabled)
<code>ui.statusline.select</code>	Statusline mode during select mode (only if <code>editor.color-modes</code> is enabled)
<code>ui.statusline.separator</code>	Separator character in statusline
<code>ui.bufferline</code>	Style for the buffer line
<code>ui.bufferline.active</code>	Style for the active buffer in buffer line
<code>ui.bufferline.background</code>	Style for bufferline background
<code>ui.popup</code>	Documentation popups (e.g. Space + k)
<code>ui.popup.info</code>	Prompt for multiple key options
<code>ui.picker.header</code>	Header row area in pickers with multiple columns
<code>ui.picker.header.column</code>	Column names in pickers with multiple columns
<code>ui.picker.header.column.active</code>	The column name in pickers with multiple columns where the cursor is entering into.
<code>ui.window</code>	Borderlines separating splits
<code>ui.help</code>	Description box for commands

Key	Notes
<code>ui.text</code>	Default text style, command prompts, popup text, etc.
<code>ui.text.focus</code>	The currently selected line in the picker
<code>ui.text.inactive</code>	Same as <code>ui.text</code> but when the text is inactive (e.g. suggestions)
<code>ui.text.info</code>	The key: command text in <code>ui.popup.info</code> boxes
<code>ui.text.directory</code>	Directory names in prompt completion
<code>ui.virtual.ruler</code>	Ruler columns (see the <code>editor.rulers</code> config)
<code>ui.virtual.whitespace</code>	Visible whitespace characters
<code>ui.virtual.indent-guide</code>	Vertical indent width guides
<code>ui.virtual.inlay-hint</code>	Default style for inlay hints of all kinds
<code>ui.virtual.inlay-hint.parameter</code>	Style for inlay hints of kind <code>parameter</code> (language servers are not required to set a kind)
<code>ui.virtual.inlay-hint.type</code>	Style for inlay hints of kind <code>type</code> (language servers are not required to set a kind)
<code>ui.virtual.wrap</code>	Soft-wrap indicator (see the <code>editor.soft-wrap</code> config)
<code>ui.virtual.jump-label</code>	Style for virtual jump labels
<code>ui.menu</code>	Code and command completion menus
<code>ui.menu.selected</code>	Selected autocomplete item
<code>ui.menu.scroll</code>	<code>fg</code> sets thumb color, <code>bg</code> sets track color of scrollbar
<code>ui.selection</code>	For selections in the editing area
<code>ui.selection.primary</code>	
<code>ui.highlight</code>	Highlighted lines in the picker preview

Key	Notes
<code>ui.highlight.frame</code>	Line at which debugging execution is paused at
<code>ui.cursorline.primary</code>	The line of the primary cursor (if <code>cursorline</code> is enabled)
<code>ui.cursorline.secondary</code>	The lines of any other cursors (if <code>cursorline</code> is enabled)
<code>ui.cursorcolumn.primary</code>	The column of the primary cursor (if <code>cursorcolumn</code> is enabled)
<code>ui.cursorcolumn.secondary</code>	The columns of any other cursors (if <code>cursorcolumn</code> is enabled)
<code>warning</code>	Diagnostics warning (gutter)
<code>error</code>	Diagnostics error (gutter)
<code>info</code>	Diagnostics info (gutter)
<code>hint</code>	Diagnostics hint (gutter)
<code>diagnostic</code>	Diagnostics fallback style (editing area)
<code>diagnostic.hint</code>	Diagnostics hint (editing area)
<code>diagnostic.info</code>	Diagnostics info (editing area)
<code>diagnostic.warning</code>	Diagnostics warning (editing area)
<code>diagnostic.error</code>	Diagnostics error (editing area)
<code>diagnostic.unnecessary</code>	Diagnostics with unnecessary tag (editing area)
<code>diagnostic.deprecated</code>	Diagnostics with deprecated tag (editing area)
<code>tabstop</code>	Snippet placeholder

Key remapping

Helix currently supports one-way key remapping through a simple TOML configuration file. (More powerful solutions such as rebinding via commands will be available in the future).

There are three kinds of commands that can be used in keymaps:

- Static commands: commands like `move_char_right` which are usually bound to keys and used for movement and editing. A list of static commands is available in the `Keymap` documentation and in the source code in `helix-term/src/commands.rs` at the invocation of `static_commands!` macro.
- Typable commands: commands that can be executed from command mode (`:`), for example `:write!`. See the `Commands` documentation for a list of available typeable commands or the `TypableCommandList` declaration in the source code at `helix-term/src/commands/typed.rs`.
- Macros: sequences of keys that are executed in order. These keybindings start with `@` and then list any number of keys to be executed. For example `@miw` can be used to select the surrounding word. For now, macro keybindings are not allowed in keybinding sequences due to limitations in the way that command sequences are executed. Modifier keys (e.g. `Alt+o`) can be used like `"<A-o>"`, e.g. `"@miw<A-o>"`

To remap keys, create a `config.toml` file in your `helix` configuration directory (default `~/.config/helix` on Linux systems) with a structure like this:

💡 To set a modifier + key as a keymap, type `A-X = ...` or `C-X = ...` for `Alt + X` or `Ctrl + X`. Combine with `Shift` using a dash, e.g. `c-s-esc`. Within macros, wrap them in `<>`, e.g. `<A-X>` and `<C-X>` to distinguish from the `A` or `C` keys.

```
# At most one section each of 'keys.normal', 'keys.insert' and 'keys.select'
[keys.normal]
C-s = ":w" # Maps Ctrl-s to the typable command :w which is an alias for
:write (save file)
C-o = ":open ~/.config/helix/config.toml" # Maps Ctrl-o to opening of the
helix config file
a = "move_char_left" # Maps the 'a' key to the move_char_left command
w = "move_line_up" # Maps the 'w' key move_line_up
"C-S-esc" = "extend_line" # Maps Ctrl-Shift-Escape to extend_line
g = { a = "code_action" } # Maps `ga` to show possible code actions
"ret" = ["open_below", "normal_mode"] # Maps the enter key to open_below then
re-enter normal mode
"A-x" = "@x<A-d>" # Maps Alt-x to a macro selecting the whole line and
deleting it without yanking it

[keys.insert]
"A-x" = "normal_mode" # Maps Alt-X to enter normal mode
j = { k = "normal_mode" } # Maps `jk` to exit insert mode
```

Minor modes

Minor modes are accessed by pressing a key (usually from normal mode), giving access to dedicated bindings. Bindings can be modified or added by nesting definitions.

```
[keys.insert.j]
k = "normal_mode" # Maps `jk` to exit insert mode

[keys.normal.g]
a = "code_action" # Maps `ga` to show possible code actions

# invert `j` and `k` in view mode
[keys.normal.z]
j = "scroll_up"
k = "scroll_down"

# create a new minor mode bound to `+`
[keys.normal."+"]
m = ":run-shell-command make"
c = ":run-shell-command cargo build"
t = ":run-shell-command cargo test"
```

Special keys and modifiers

Ctrl, Shift and Alt modifiers are encoded respectively with the prefixes `C-`, `S-` and `A-`.

The **Super** key - the Windows/Linux key or the Command key on Mac keyboards - is also supported when using a terminal emulator that supports the **enhanced keyboard protocol**. The super key is encoded with prefixes `Meta-`, `Cmd-` or `Win-`. These are all synonyms for the super modifier - binding a key with a `Win-` modifier will mean it can be used with the Windows/Linux key or the Command key.

```
[keys.normal]
C-s = ":write" # Ctrl and 's' to write
Cmd-s = ":write" # Cmd or Win or Meta and 's' to write
```

Special keys are encoded as follows:

Key name	Representation
Backspace	"backspace"
Space	"space"
Return/Enter	"ret"
Left	"left"
Right	"right"
Up	"up"
Down	"down"
Home	"home"
End	"end"
Page Up	"pageup"
Page Down	"pagedown"
Tab	"tab"
Delete	"del"
Insert	"ins"
Null	"null"

Key name	Representation
----------	----------------

Escape	"esc"
--------	-------

Keys can be disabled by binding them to the `no_op` command.

All other keys such as `?`, `!`, `-` etc. can be used literally:

```
[keys.normal]
"?" = ":write"
"!" = ":write"
"_" = ":write"
```

Note: `-` can't be used when combined with a modifier, for example `Alt + -` should be written as `A-minus`. `A--` is not accepted.

Languages

Language-specific settings and settings for language servers are configured in `languages.toml` files.

languages.toml files

There are three possible locations for a `languages.toml` file:

1. In the Helix source code, which lives in the `Helix repository`. It provides the default configurations for languages and language servers.
2. In your `configuration directory`. This overrides values from the built-in language configuration. For example, to disable auto-formatting for Rust:

```
# in <config_dir>/helix/languages.toml

[language-server.mylang-lsp]
command = "mylang-lsp"

[[language]]
name = "rust"
auto-format = false
```

3. In a `.helix` folder in your project. Language configuration may also be overridden local to a project by creating a `languages.toml` file in a `.helix` folder. Its settings will be merged with the language configuration in the configuration directory and the built-in configuration.

Language configuration

Each language is configured by adding a `[[language]]` section to a `languages.toml` file. For example:

```
[[language]]
name = "mylang"
scope = "source.mylang"
injection-regex = "mylang"
file-types = ["mylang", "myl"]
comment-tokens = "#"
indent = { tab-width = 2, unit = "  " }
formatter = { command = "mylang-formatter" , args = ["--stdin"] }
language-servers = [ "mylang-lsp" ]
```

These configuration keys are available:

Key	Description
<code>name</code>	The name of the language
<code>language-id</code>	The language-id for language servers, checkout the table at <code>TextDocumentItem</code> for the right id
<code>scope</code>	A string like <code>source.js</code> that identifies the language. Currently, we strive to match the scope names used by popular TextMate grammars and by the Linguist library. Usually <code>source.<name></code> or <code>text.<name></code> in case of markup languages
<code>injection-regex</code>	regex pattern that will be tested against a language name in order to determine whether this language should be used for a potential language injection site.
<code>file-types</code>	The filetypes of the language, for example <code>["yaml", "yml"]</code> . See the file-type detection section below.
<code>shebangs</code>	The interpreters from the shebang line, for example <code>["sh", "bash"]</code>
<code>roots</code>	A set of marker files to look for when trying to find the workspace root. For example <code>Cargo.lock</code> , <code>yarn.lock</code>
<code>auto-format</code>	Whether to autoformat this language when saving
<code>diagnostic-severity</code>	Minimal severity of diagnostic for it to be displayed. (Allowed values: <code>error</code> , <code>warning</code> , <code>info</code> , <code>hint</code>)
<code>comment-tokens</code>	The tokens to use as a comment token, either a single token <code>"//"</code> or an array <code>["//", "///", "///!"]</code> (the first token will be used for commenting). Also configurable as <code>comment-token</code> for backwards compatibility

Key	Description
<code>block-comment-tokens</code>	The start and end tokens for a multiline comment either an array or single table of <code>{ start = "/*", end = "*/" }</code> . The first set of tokens will be used for commenting, any pairs in the array can be uncommented
<code>indent</code>	The indent to use. Has sub keys <code>unit</code> (the text inserted into the document when indenting; usually set to N spaces or <code>"\t"</code> for tabs) and <code>tab-width</code> (the number of spaces rendered for a tab)
<code>language-servers</code>	The Language Servers used for this language. See below for more information in the section Configuring Language Servers for a language
<code>grammar</code>	The tree-sitter grammar to use (defaults to the value of <code>name</code>)
<code>formatter</code>	The formatter for the language, it will take precedence over the lsp when defined. The formatter must be able to take the original file as input from stdin and write the formatted file to stdout. The filename of the current buffer can be passed as argument by using the <code>%{buffer_name}</code> expansion variable. See below for more information in the Configuring the formatter command
<code>soft-wrap</code>	<code>editor.softwrap</code>
<code>text-width</code>	Maximum line length. Used for the <code>:reflow</code> command and soft-wrapping if <code>soft-wrap.wrap-at-text-width</code> is set, defaults to <code>editor.text-width</code>
<code>rulers</code>	Overrides the <code>editor.rulers</code> config key for the language.
<code>path-completion</code>	Overrides the <code>editor.path-completion</code> config key for the language.
<code>workspace-lsp-roots</code>	Directories relative to the workspace root that are treated as LSP roots. Should only be set in <code>.helix/config.toml</code> . Overwrites the setting of the same name in <code>config.toml</code> if set.
<code>persistent-diagnostic-sources</code>	An array of LSP diagnostic sources assumed unchanged when the language server resends the same set of diagnostics. Helix can track the position for these diagnostics internally instead. Useful for diagnostics that are recomputed on save.

File-type detection and the file-types key

Helix determines which language configuration to use based on the `file-types` key from the above section. `file-types` is a list of strings or tables, for example:

```
file-types = ["toml", { glob = "Makefile" }, { glob = ".git/config" }, { glob = ".github/workflows/*.yaml" } ]
```

When determining a language configuration to use, Helix searches the file-types with the following priorities:

1. Glob: values in `glob` tables are checked against the full path of the given file. Globs are standard Unix-style path globs (e.g. the kind you use in Shell) and can be used to match paths for a specific prefix, suffix, directory, etc. In the above example, the `{ glob = "Makefile" }` config would match files with the name `Makefile`, the `{ glob = ".git/config" }` config would match `config` files in `.git` directories, and the `{ glob = ".github/workflows/*.yaml" }` config would match any `yaml` files in `.github/workflow` directories. Note that globs should always use the Unix path separator `/` even on Windows systems; the matcher will automatically take the machine-specific separators into account. If the glob isn't an absolute path or doesn't already start with a glob prefix, `*/` will automatically be added to ensure it matches for any subdirectory.
2. Extension: if there are no glob matches, any `file-types` string that matches the file extension of a given file wins. In the example above, the `"toml"` config matches files like `Cargo.toml` or `languages.toml`.

Configuring the formatter command

Command line expansions are supported in the arguments of the formatter command. In particular, the `%{buffer_name}` variable can be passed as argument to the formatter:

```
formatter = { command = "mylang-formatter" , args = ["--stdin", "--stdin-filename %{buffer_name}"] }
```

Language Server configuration

Language servers are configured separately in the table `language-server` in the same file as the languages `languages.toml`

For example:

```
[language-server.mylang-lsp]
command = "mylang-lsp"
args = ["--stdio"]
config = { provideFormatter = true }
environment = { "ENV1" = "value1", "ENV2" = "value2" }

[language-server.efm-lsp-prettier]
command = "efm-langserver"

[language-server.efm-lsp-prettier.config]
documentFormatting = true
languages = { typescript = [ { formatCommand = "prettier --stdin-filepath
${INPUT}", formatStdin = true } ] }
```

These are the available options for a language server.

Key	Description
<code>command</code>	The name or path of the language server binary to execute. Binaries must be in <code>\$PATH</code>
<code>args</code>	A list of arguments to pass to the language server binary
<code>config</code>	Language server initialization options
<code>timeout</code>	The maximum time a request to the language server may take, in seconds. Defaults to <code>20</code>
<code>environment</code>	Any environment variables that will be used when starting the language server <code>{ "KEY1" = "Value1", "KEY2" = "Value2" }</code>
<code>required-root-patterns</code>	A list of <code>glob</code> patterns to look for in the working directory. The language server is started if at least one of them is found.

A `format` sub-table within `config` can be used to pass extra formatting options to Document Formatting Requests. For example, with typescript:

```
[language-server.typescript-language-server]
# pass format options according to https://github.com/typescript-language-
server/typescript-language-server#workspacedidchangeconfiguration omitting the
"[language].format." prefix.
config = { format = { "semicolons" = "insert",
"insertSpaceBeforeFunctionParenthesis" = true } }
```

Configuring Language Servers for a language

The `language-servers` attribute in a language tells helix which language servers are used for this language.

They have to be defined in the `[language-server]` table as described in the previous section.

Different languages can use the same language server instance, e.g. `typescript-language-server` is used for javascript, jsx, tsx and typescript by default.

The definition order of language servers affects the order in the results list of code action menu.

In case multiple language servers are specified in the `language-servers` attribute of a `language`, it's often useful to only enable/disable certain language-server features for these language servers.

As an example, `efm-lsp-prettier` of the previous example is used only with a formatting command `prettier`, so everything else should be handled by the `typescript-language-server` (which is configured by default). The language configuration for typescript could look like this:

```
[[language]]
name = "typescript"
language-servers = [ { name = "efm-lsp-prettier", only-features = [ "format" ]
}, "typescript-language-server" ]
```

or equivalent:

```
[[language]]
name = "typescript"
language-servers = [ { name = "typescript-language-server", except-features =
[ "format" ] }, "efm-lsp-prettier" ]
```


Each requested LSP feature is prioritized in the order of the `language-servers` array. For example, the first `goto-definition` supported language server (in this case `typescript-language-server`) will be taken for the relevant LSP request (command `goto_definition`). The features `diagnostics`, `code-action`, `completion`, `document-symbols` and `workspace-symbols` are an exception to that rule, as they are working for all language servers at the same time and are merged together, if enabled for the language. If no `except-features` or `only-features` is given, all features for the language server are enabled. If a language server itself doesn't support a feature, the next language server array entry will be tried (and so on).

The list of supported features is:

- `format`
- `goto-definition`
- `goto-declaration`
- `goto-type-definition`
- `goto-reference`
- `goto-implementation`
- `signature-help`
- `hover`
- `document-highlight`
- `completion`
- `code-action`
- `workspace-command`
- `document-symbols`
- `workspace-symbols`
- `diagnostics`
- `rename-symbol`
- `inlay-hints`

Tree-sitter grammar configuration

The source for a language's tree-sitter grammar is specified in a `[[grammar]]` section in `languages.toml`. For example:

```
[[grammar]]
name = "mylang"
source = { git = "https://github.com/example/mylang", rev =
"a250c4582510ff34767ec3b7dcdd3c24e8c8aa68" }
```

Grammar configuration takes these keys:

Key	Description
<code>name</code>	The name of the tree-sitter grammar
<code>source</code>	The method of fetching the grammar - a table with a schema defined below

Where `source` is a table with either these keys when using a grammar from a git repository:

Key	Description
<code>git</code>	A git remote URL from which the grammar should be cloned
<code>rev</code>	The revision (commit hash or tag) which should be fetched
<code>subpath</code>	A path within the grammar directory which should be built. Some grammar repositories host multiple grammars (for example <code>tree-sitter-typescript</code> and <code>tree-sitter-ocaml</code>) in subdirectories. This key is used to point <code>hx --grammar build</code> to the correct path for compilation. When omitted, the root of repository is used

Choosing grammars

You may use a top-level `use-grammars` key to control which grammars are fetched and built when using `hx --grammar fetch` and `hx --grammar build`.

```
# Note: this key must come before the [[language]] and [[grammar]]
sections
use-grammars = { only = [ "rust", "c", "cpp" ] }
# or
use-grammars = { except = [ "yaml", "json" ] }
```

When omitted, all grammars are fetched and built.

Guides

This section contains guides for adding new language server configurations, tree-sitter grammars, textobject queries, and other similar items.

Adding new languages to Helix

In order to add a new language to Helix, you will need to follow the steps below.

Language configuration

1. Add a new `[[language]]` entry in the `languages.toml` file and provide the necessary configuration for the new language. For more information on language configuration, refer to the [language configuration](#) section of the documentation. A new language server can be added by extending the `[language-server]` table in the same file.
2. If you are adding a new language or updating an existing language server configuration, run the command `cargo xtask docgen` to update the [Language Support](#) documentation.

💡 If you are adding a new Language Server configuration, make sure to update the [Language Server Wiki](#) with the installation instructions.

Grammar configuration

1. If a tree-sitter grammar is available for the new language, add a new `[[grammar]]` entry to the `languages.toml` file.
2. If you are testing the grammar locally, you can use the `source.path` key with an absolute path to the grammar. However, before submitting a pull request, make sure to switch to using `source.git`.

Queries

1. In order to provide syntax highlighting and indentation for the new language, you will need to add queries.
2. Create a new directory for the language with the path `runtime/queries/<name>/`.
3. Refer to the [tree-sitter website](#) for more information on writing

queries.

4. A list of highlight captures can be found on the [themes page](#).

Common issues

- If you encounter errors when running Helix after switching branches, you may need to update the tree-sitter grammars. Run the command `hx --grammar fetch` to fetch the grammars and `hx --grammar build` to build any out-of-date grammars.
- If a parser is causing a segfault, or you want to remove it, make sure to remove the compiled parser located at `runtime/grammars/<name>.so`.
- If you are attempting to add queries and Helix is unable to locate them, ensure that the environment variable `HELIX_RUNTIME` is set to the location of the `runtime` folder you're developing in.

Adding textobject queries

Helix supports textobjects that are language specific, such as functions, classes, etc. These textobjects require an accompanying tree-sitter grammar and a `textobjects.scm` query file to work properly. Tree-sitter allows us to query the source code syntax tree and capture specific parts of it. The queries are written in a lisp dialect. More information on how to write queries can be found in the [official tree-sitter documentation](#).

Query files should be placed in `runtime/queries/{language}/textobjects.scm` when contributing to Helix. Note that to test the query files locally you should put them under your local runtime directory (`~/.config/helix/runtime` on Linux for example).

The following captures are recognized:

Capture Name

`function.inside`

`function.around`

`class.inside`

`class.around`

`test.inside`

`test.around`

`parameter.inside`

`parameter.around`

`comment.inside`

`comment.around`

`entry.inside`

`entry.around`

Example query files can be found in the [helix GitHub repository](#).

Queries for textobject based navigation

Tree-sitter based navigation in Helix is done using captures in the following order:

- `object.movement`
- `object.around`
- `object.inside`

For example if a `function.around` capture has been already defined for a language in its `textobjects.scm` file, function navigation should also work automatically. `function.movement` should be defined only if the node captured by `function.around` doesn't make sense in a navigation context.

Adding indent queries

Helix uses tree-sitter to correctly indent new lines. This requires a tree-sitter grammar and an `indent.scm` query file placed in `runtime/queries/{language}/indents.scm`. The indentation for a line is calculated by traversing the syntax tree from the lowest node at the beginning of the new line (see [Indent queries](#)). Each of these nodes contributes to the total indent when it is captured by the query (in what way depends on the name of the capture).

Note that it matters where these added indents begin. For example, multiple indent level increases that start on the same line only increase the total indent level by 1. See [Capture types](#).

By default, Helix uses the `hybrid` indentation heuristic. This means that indent queries are not used to compute the expected absolute indentation of a line but rather the expected difference in indentation between the new and an already existing line. This difference is then added to the actual indentation of the already existing line. Since this makes errors in the indent queries harder to find, it is recommended to disable it when testing via `:set indent-heuristic tree-sitter`. The rest of this guide assumes that the `tree-sitter` heuristic is used.

Indent queries

When Helix is inserting a new line through `o`, `O`, or `<ret>`, to determine the indent level for the new line, the query in `indents.scm` is run on the document. The starting position of the query is the end of the line above where a new line will be inserted.

For `o`, the inserted line is the line below the cursor, so that starting position of the query is the end of the current line.


```
fn need_hero(some_hero: Hero, life: Life) -> {
  matches!(some_hero, Hero { // ←
    strong: true, // ←
    fast: true,   // ←
    sure: true,   // ←
    soon: true,   // ←
  }) &&          // ←
  // ↑
  // ↑
  some_hero > life
}
```

For `0`, the newly inserted line is the *current* line, so the starting position of the query is the end of the line above the cursor.

```
fn need_hero(some_hero: Hero, life: Life) -> { // ←
  matches!(some_hero, Hero { // ←
    strong: true, // ←
    fast: true,   // ←
    sure: true,   // ←
    soon: true,   // ←
  }) &&          // ←
  some_hero > life // ←
} // ←
```

From this starting node, the syntax tree is traversed up until the root node. Each indent capture is collected along the way, and then combined according to their `capture types` and `scopes` to a final indent level for the line.

Capture types

- `@indent` (default scope `tail`): Increase the indent level by 1. Multiple occurrences in the same line *do not* stack. If there is at least one `@indent` and one `@outdent` capture on the same line, the indent level isn't changed at all.
- `@outdent` (default scope `all`): Decrease the indent level by 1. The same rules as for `@indent` apply.
- `@indent.always` (default scope `tail`): Increase the indent level by 1. Multiple occurrences on the same line *do* stack. The final indent level is `@indent.always` - `@outdent.always`. If an `@indent` and an `@indent.always` are on the same line, the `@indent` is ignored.
- `@outdent.always` (default scope `all`): Decrease the indent level by

- @indent / @outdent

5/24/26, 8:34 PM

Note how on the second line, we have two blocks begin on the same line. In this case, since both captures occur on the same line, they are combined and only result in a net increase of 1. Also note that the closing `}`s are part of the `@indent` captures, but the 3 `@outdent`s also combine into 1 and result in that line losing one indent level.

`@extend` / `@extend.prevent-once`

For an example of where `@extend` can be useful, consider Python, which is whitespace-sensitive.

```
]
(parenthesized_expression)
(function_definition)
(class_definition)
] @indent
```

```
class Hero:
    def __init__(self, strong, fast, sure, soon):#
        self.is_strong = strong #
        self.is_fast = fast      # query start
        self.is_sure = sure      # cursor
        self.is_soon = soon      #
        #
        #
        #
        #
    def need_hero(self, life):      #
        return (                  #
            self.is_strong         #
            and self.is_fast       #
            and self.is_sure       #
            and self.is_soon       #
            and self > life        #
        ) # ← traversal
                                start node
```

Without braces to catch the scope of the function, the smallest descendant of the cursor on a line feed ends up being the entire inside of the class. Because of this, it will miss the entire function node and its indent capture, leading to an indent level one too small.

To address this case, `@extend` tells helix to "extend" the captured node's span to the line feed and every consecutive line that has a greater indent level than the line of the node.

```
(parenthesized_expression) @indent

]
(function_definition)
(class_definition)
] @indent @extend
```

```
class Hero:
    def __init__(self, strong, fast, sure, soon):#
        self.is_strong = strong #
        self.is_fast = fast      #
        self.is_sure = sure      #
        self.is_soon = soon      #
        #
        #
        #
    def need_hero(self, life):
        return (
            self.is_strong
            and self.is_fast
            and self.is_sure
            and self.is_soon
            and self > life
        )
```

The diagram illustrates the cursor's position and movement within the `need_hero` function. A bracket on the right side of the function body is labeled "traversal start node". An arrow points from this bracket to the first line of the function body, `self.is_strong = strong`. Another arrow points from the same bracket to the last line of the function body, `and self > life`. A third arrow points from the bracket to the line `self.is_soon = soon`. A fourth arrow points from the bracket to the line `#` immediately following `self.is_soon = soon`. A fifth arrow points from the bracket to the line `#` immediately following the `return` statement. A sixth arrow points from the bracket to the line `#` immediately following the `def` statement.

Furthermore, there are some cases where extending to everything with a greater indent level may not be desirable. Consider the `need_hero` function above. If our cursor is on the last line of the returned expression.

```
class Hero:
    def __init__(self, strong, fast, sure, soon):
        self.is_strong = strong
        self.is_fast = fast
        self.is_sure = sure
        self.is_soon = soon

    def need_hero(self, life):
        return (
            self.is_strong
            and self.is_fast
            and self.is_sure
            and self.is_soon
            and self > life
        ) # ← cursor
# ←————— where cursor should go on new line
```

In Python, there are a few tokens that will always end a scope, such as a return statement. Since the scope ends, so should the indent level. But because the function span is extended to every line with a greater indent level, a new line would just continue on the same level. And an `@outdent` would not help us here either, since it would cause everything in the parentheses to become outdented as well.

To help, we need to signal an end to the extension. We can do this with `@extend.prevent-once`.

```
(parenthesized_expression) @indent
]
  (function_definition)
  (class_definition)
] @indent @extend

(return_statement) @extend.prevent-once
```

`@indent.always` / `@outdent.always`

As mentioned before, normally if there is more than one `@indent` or `@outdent` capture on the same line, they are combined.

Sometimes, there are cases when you may want to ensure that every indent capture is additive, regardless of how many occur on the same line. Consider this example in YAML.

```
- foo: bar
# ↑ ↑
# |   start of map
# |   start of list element
# |   |
# |   |   baz: quux # ← cursor
# |   |   # ← where the cursor should go on a new line
# |   |   garply: waldo
# |   - quux:
# |       bar: baz
# |       xyzy: thud
# |       fred: plugh
```

In YAML, you often have lists of maps. In these cases, the syntax is such that the list element and the map both start on the same line. But we really do want to start an indentation for each of these so that subsequent keys in the map hang over the list and align properly. This

is where `@indent.always` helps.

```
((block_sequence_item) @item @indent.always @extend
  (#not-one-line? @item))

((block_mapping_pair
  key: (_) @key
  value: (_) @val
  (#not-same-line? @key @val)
) @indent.always @extend
)
```

Predicates

In some cases, an S-expression cannot express exactly what pattern should be matched. For that, tree-sitter allows for predicates to appear anywhere within a pattern, similar to how `#set!` declarations work:

```
(some_kind
  (child_kind) @indent
  (#predicate? arg1 arg2 ...)
)
```

The number of arguments depends on the predicate that's used. Each argument is either a capture (`@name`) or a string (`"some string"`). The following predicates are supported by tree-sitter:

- `#eq?` / `#not-eq?`: The first argument (a capture) must/must not be equal to the second argument (a capture or a string).
- `#match?` / `#not-match?`: The first argument (a capture) must/must not match the regex given in the second argument (a string).
- `#any-of?` / `#not-any-of?`: The first argument (a capture) must/must not be one of the other arguments (strings).

Additionally, we support some custom predicates for indent queries:

- `#not-kind-eq?`: The kind of the first argument (a capture) must not be equal to the second argument (a string).
- `#same-line?` / `#not-same-line?`: The captures given by the 2 arguments must/must not start on the same line.

- `#one-line?` / `#not-one-line?`: The captures given by the first argument must/must not span a total of one line.

Scopes

Added indents don't always apply to the whole node. For example, in most cases when a node should be indented, we actually only want everything except for its first line to be indented. For this, there are several scopes (more scopes may be added in the future if required):

- `tail`: This scope applies to everything except for the first line of the captured node.
- `all`: This scope applies to the whole captured node. This is only different from `tail` when the captured node is the first node on its line.

For example, imagine we have the following function

```
fn aha() { // ←
  let take = "on me"; // ←
  let take = "me on"; //
  let ill = be_gone_days(1 || 2); //
} // ←
```

(block) @indent
"}" @outdent
scope: "all"

We can write the following query with the `#set!` declaration:

```
((block) @indent
  (#set! "scope" "tail"))
("}" @outdent
  (#set! "scope" "all"))
```

As we can see, the "tail" scope covers the node, except for the first line. Everything up to and including the closing brace gets an indent level of 1. Then, on the closing brace, we encounter an outdent with a scope of "all", which means the first line is included, and the indent level is cancelled out on this line. (Note these scopes are the defaults for `@indent` and `@outdent`—they are written explicitly for demonstration.)

Adding Injection Queries

Writing language injection queries allows one to highlight a specific node as a different language. In addition to the `standard` language injection options used by tree-sitter, there are a few Helix specific extensions that allow for more control.

An example of a simple query that would highlight all strings as bash in Nix:

```
((string_expression (string_fragment) @injection.content)
  (#set! injection.language "bash"))
```

Another example is this query, which highlights links in comments and keywords like "TODO", by reusing the dedicated "comment" language:

```
((comment) @injection.content
  (#set! injection.language "comment"))
```

Capture Types

- `@injection.language` (standard): The captured node may contain the language name used to highlight the node captured by `@injection.content`.
- `@injection.content` (standard): Marks the content to be highlighted as the language captured with `@injection.language` *et al*.
- `@injection.filename` (extension): The captured node may contain a filename with a file-extension known to Helix, highlighting `@injection.content` as that language. This uses the language extensions defined in both the default `languages.toml` distributed with Helix, as well as user defined languages.
- `@injection.shebang` (extension): The captured node may contain a shebang used to choose a language to highlight as. This also uses the shebangs defined in the default and user `languages.toml`.

Settings

- `injection.combined` (standard): Indicates that all the matching nodes in the tree should have their content parsed as one nested document.
- `injection.language` (standard): Forces the captured content to be highlighted as the given language
- `injection.include-children` (standard): Indicates that the content node's entire text should be re-parsed, including the text of its child nodes. By default, child nodes' text will be excluded from the injected document.
- `injection.include-unnamed-children` (extension): Same as `injection.include-children` but only for unnamed child nodes.

Predicates

- `#eq?` (standard): The first argument (a capture) must be equal to the second argument (a capture or a string).
- `#match?` (standard): The first argument (a capture) must match the regex given in the second argument (a string).
- `#any-of?` (standard): The first argument (a capture) must be one of the other arguments (strings).